

**ANALISIS PERBANDINGAN MODEL
SEQUENTIAL NEURAL NETWORK PADA
SETIAP TIER PADA PASAR MODAL
INDONESIA**

Proposal Tugas Akhir

Oleh

Jonathan Arthurito Aldi Sinaga

NIM 18221079



**PROGRAM STUDI SISTEM DAN TEKNOLOGI
INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

Desember 2024

**ANALISIS PERBANDINGAN MODEL
SEQUENTIAL NEURAL NETWORK PADA
SETIAP TIER PADA PASAR MODAL
INDONESIA**

Proposal Tugas Akhir

Oleh

Jonathan Arthurito Aldi Sinaga

NIM 18221079

Program Studi Sistem dan Teknologi Informasi

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Telah disetujui dan disahkan sebagai Laporan Tugas Akhir
, di Bandung, pada tanggal

Pembimbing I

Dr. Ir. Dimitri Mahayana, M.Eng.

NIP 196808091991021001

Daftar Isi

I	Pendahuluan	1
I.1	Latar Belakang	1
I.2	Rumusan Masalah	4
I.3	Tujuan	5
I.4	Batasan Masalah	5
I.5	Metodologi	6
II	Studi Literatur	8
II.1	Pasar Saham Indonesia	8
II.2	Kecerdasan Buatan	11
II.3	Pembelajaran Mesin	12
II.4	Time Series data	13
II.5	Neural Network	14
II.5.1	Recurrent Neural Network	18
II.5.2	Transformer	27
II.6	Evaluation Metrics	33
II.6.1	Root Mean Squared Error (RMSE)	33
II.6.2	Directional Accuracy (DA)	33
II.6.3	Mean Absolute Percentage Error (MAPE)	34
II.6.4	Coefficient of Determination (R^2)	34
II.7	Penelitian Terkait	35
II.7.1	<i>Artificial Neural Network for Predicting Indonesia Stock Exchange Composite Using Macroeconomic Variables</i> (Alam- syah and Zahir 2018)	35

II.7.2	<i>Indonesian Banking Stock Price Prediction with LSTM and Random Walk Method</i> (Heru, Rachmawati, and Suhartono 2021)	36
III	Analisis dan Perancangan	38
III.1	Analisis Masalah	38
III.2	Identifikasi Alternatif Solusi	38
III.2.1	<i>Classical Quantitative</i>	38
III.2.2	<i>Sequence-based Modelling</i>	39
III.2.3	<i>Classical Machine Learning</i>	40
III.3	Evaluasi dan Pemilihan Alternatif Solusi	41
III.4	Perancangan Konsep Desain dan Solusi	43
III.4.1	Data Preparation	43
III.4.2	Data Understanding	45
III.4.3	Rancangan Konsep Solusi	49
IV	Rencana Selanjutnya	53
IV.1	Rencana Implementasi	53
IV.1.1	Perkakas	53
IV.1.2	Biaya	53
IV.1.3	Rencana Implementasi	54
IV.2	Desain Pengujian dan Evaluasi	54
IV.3	Analisis Mitigasi dan Risiko	55
IV.3.1	<i>Timeline</i> yang Terlewat	55
IV.3.2	Hasil Pemodelan yang Kurang Baik	55
IV.3.3	Pemodelan Memakan Waktu yang Cukup Lama	55

Daftar Gambar

I.1.1 Peningkatan Jumlah Investor di Indonesia (Asiavesta 2022)	2
I.1.2 Peningkatan Jumlah Perusahaan di Pasar Modal (Asiavesta 2022)	2
I.5.1 Metodologi CRISP-DM. Sumber: (Vorhies 2016)	6
II.5.1 <i>Biological neuron</i> (Orr et al. 2021)	14
II.5.2 Graf <i>Neural Network</i> . (Waskiewicz 2021)	15
II.5.3 Perceptron buatan (Haykin 1998)	16
II.5.4 Data non-linear (Learning 2021)	17
II.5.5 Arsitektur RNN (S November 2024)	18
II.5.6 Diagram LSTM (Olah 2015)	22
II.5.7 Diagram Sel GRU (Bibi et al. 2020)	25
II.5.8 Arsitektur <i>Transformer</i> (Vaswani et al. 2017)	28
III.4.1 Distribusi <i>Adj. Close</i>	47
III.4.2 Distribusi <i>Closing price</i>	47
III.4.3 Distribusi <i>High price</i>	48
III.4.4 Distribusi <i>low price</i>	48
III.4.5 Distribusi <i>opening price</i>	49
III.4.6 Distribusi volume penjualan	49
III.4.7 Diagram Konsep Solusi	52

Daftar Tabel

III.3. Perbandingan Kelebihan dan Kekurangan Alternatif Solusi	42
III.3.2 Weighted Scoring Matrix Alternatif Solusi	43
III.4. Metadata of Stock Market Time Series Dataset	44
III.4.2 Missing Value Analysis for Stock Market Data	45
III.4.3 Descriptive Statistics for Stock Market Data	46
IV.1. Spesifikasi Perangkat	53
IV.1.2 Rincian Biaya	54
IV.1.3 Linimasa pengerjaan tahap 1	54
IV.1.4 Linimasa pengerjaan tahap 2	54

Daftar Persamaan

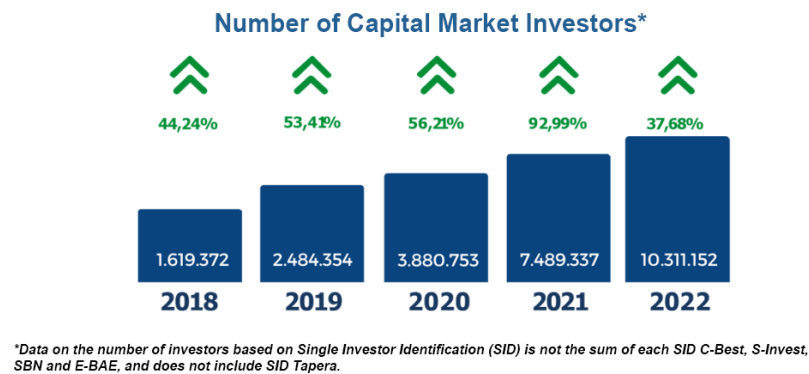
II.1	<i>Summing Function</i>	16
II.2	<i>Rectified Linear Unit Function</i>	17
II.3	<i>Mean Squared Error</i>	17
II.4	<i>Hidden state pada Vanilla RNN</i>	19
II.5	<i>Output pada Vanilla RNN</i>	19
II.6	Persamaan <i>positional encoding</i>	29
II.7	<i>Scaled Dot-Product Attention</i>	29
II.8	<i>Residual connection & multi-head self-attention</i>	30
II.9	<i>Residual connection & feed-forward network</i>	30
II.10	<i>Root Mean Squared Error</i>	33
II.11	<i>Directional Accuracy</i>	34
II.12	<i>Mean Absolute Percentage Error</i>	34
II.13	<i>Coefficient of Determination (R^2)</i>	35

BAB I

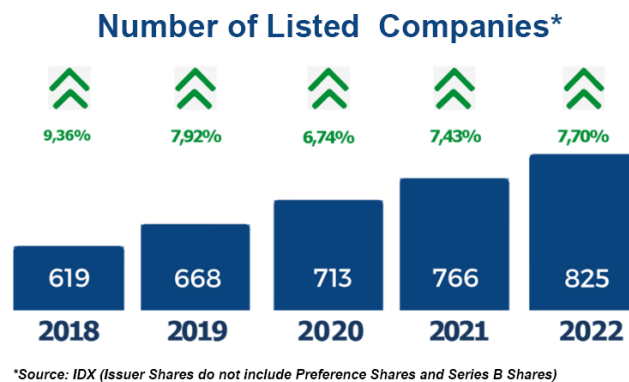
PENDAHULUAN

I.1 Latar Belakang

Pasar modal Indonesia telah mengalami transformasi signifikan selama beberapa dekade terakhir ini. Pasar yang mulanya berskala kecil dengan jumlah emiten yang terbatas (OJK 2007) kini berkembang menjadi gelanggang investasi yang lebih besar, lebih terdiversifikasi, dan lebih dikenal dalam kancah internasional (OECD 2020). Pembentukan Bursa Efek Indonesia (BEI) pada tahun 2007, yang merupakan penggabungan Bursa Efek Jakarta dan Bursa Efek Semarang, dan reformasi regulasi yang dilakukan oleh Otoritas Jasa Keuangan (OJK) menjadi salah satu batu loncatan dalam perkembangan pasar modal Indonesia. Kepercayaan investor, baik lokal maupun asing, semakin kuat (Yuliani, Isnurhadi, and Jie 2017), dan jumlah perusahaan dari berbagai sektor yang menawarkan sahamnya kian meningkat. , dan jumlah perusahaan dari berbagai sektor yang menawarkan sahamnya kian meningkat.



Gambar I.1.1: Peningkatan Jumlah Investor di Indonesia (Asiavesta 2022)



Gambar I.1.2: Peningkatan Jumlah Perusahaan di Pasar Modal (Asiavesta 2022)

Pertumbuhan ekonomi yang pesat dan upaya pembangunan infrastruktur yang masif turut menjadikan Indonesia target menarik bagi investor yang mencari peluang di pasar negara berkembang (WB 2023). Bertumbuhnya ekonomi didampingi dengan masifnya upaya pembangunan infrastruktur di Indonesia menjadi faktor dalam menciptakan ekosistem bisnis yang positif. Akan tetapi, dinamika pasar Indonesia juga kompleks. Harga saham dipengaruhi oleh berbagai faktor seperti kebijakan pemerintah, indikator makroekonomi (IMF 2022), sentimen publik (Fekrazad, Harun, and Sardar 2022), harga komoditas global (FAO 2020), serta perubahan kondisi politik baik itu domestik ataupun mancanegara (Bloom et al. 2019). Hal ini membuat prediksi harga saham secara akurat menjadi rumit dikarenakan banyaknya variabel yang harus dikonsiderasi yang juga bersifat

dinamik.

Model statistik tradisional seperti ARIMA, VAR, ataupun GARCH telah lama digunakan untuk melakukan prediksi harga saham (Napitupulu and Wijaya 2013). Namun, model-model konvensional ini cenderung melihat hubungan antarvariabel secara linear dan tidak dapat mengakomodasi pola yang rumit dan dinamis (Goodfellow, Bengio, and Courville 2016). Data keuangan biasanya tidak hanya dipengaruhi oleh satu ataupun dua vektor, melainkan berbagai variabel, yang mungkin saling independen atau co-independen, yang tidak memiliki karakteristik linearitas antar keduanya tersebut (Bhardwaj and Kwatra 2022). Hal ini membuat dibutuhkan sebuah pendekatan yang dapat lebih fleksibel ataupun adaptif terhadap dinamika-dinamika data keuangan tersebut.

Dalam beberapa tahun terakhir, metode - metode yang digunakan untuk melakukan pelatihan pada model *Machine Learning* dan *Deep Learning* menjadi salah satu metode yang digunakan dalam melakukan pemodelan keuangan. Kekuatan utama dari teknik *Deep Learning* adalah kemampuan model dalam mencari serta mempelajari pola - pola implisit yang terdapat di dalam data yang besar dan kompleks . *Recurrent Neural Network* menjadi salah satu pilihan awal untuk melakukan pemodelan dalam *Time series data*. Namun, arsitektur yang digunakan dalam *RNN* biasanya memiliki masalah yaitu *Vanishing Gradient* (Hochreiter and Schmidhuber 1997) yang dapat membatasi kemampuan model dalam mencari pola dalam rentang waktu yang panjang.

Arsitektur *Long Short-Term Memory* (LSTM) muncul sebagai solusi yang dapat mempertahankan informasi lokal dalam skala data yang panjang dengan mekanisme *Gating* (Hochreiter and Schmidhuber 1997). Arsitektur *Transformer* yang populer melalui *paper* yang berjudul “Attention Is All You Need.” membuka cara baru dalam melakukan *modelling* pada data sekuensial (Vaswani et al. 2017). Alih - alih melakukan *data processing* secara berurutan seperti yang dilakukan oleh RNN, *Transformer* menggunakan mekanisme *attention* untuk menilai seberapa penting bagian tertentu dari data secara tepat sekuensial mengikuti urutan kronologis yang kaku.

Seiring berkembangnya arsitektur *Transformer*, muncul pula derivatif lainnya seperti *Temporal Fusion Transformer* (TFT) yang dirancang untuk dapat melakukan *multi-horizon time series prediction*. TFT tidak hanya berfokus kepada tingkat dari akurasi prediksi, melainkan juga mempermudah interpretasi atau *Model Understanding* (Lim et al. 2020)

Permasalahan utama dari prediksi pasar modal Indonesia adalah bagaimana cara memprediksi harga saham secara akurat dengan mengkonsiderasi banyak variabel yang saling independen ataupun co-independen serta perubahan dinamika pasar. Berbagai solusi telah dicoba seperti menggunakan model *traditional statistical inference* sampai menggunakan model berbasis *Deep Learning*. Pendekatan-pendekatan lainnya seperti ARIMA ataupun GARCH dapat juga menawarkan kemudahan serta interpretabilitas dari model yang sederhana namun tidak dapat menangani untuk mencari pola nonlinear. Sementara itu, model *Deep Learning* seperti LSTM dapat memberikan peningkatan performa dalam menangkap pola intrinsik yang mungkin terdapat pada data *Time series*.

Penelitian ini berfokus pada perbandingan implementasi model dengan arsitektur *Temporal Fusion Transformer* dengan implementasi model dengan arsitektur *Long Short-Term Memory* dengan varian - varian modernnya.

I.2 Rumusan Masalah

Berdasarkan latar belakang yang sudah dijelaskan sebelumnya, berikut adalah rumusan masalah yang diajukan.

1. Bagaimana cara mengimplementasikan model dengan arsitektur *Temporal Fusion Transformer* pada data *Time Series* pasar saham Indonesia ?
2. Bagaimana cara mengimplementasikan model dengan varian-varian arsitektur *Long Short-Term Memory* pada data *Time Series* pasar saham Indonesia ?
3. Bagaimana perbandingan performa kinerja antara model dengan arsitektur *Temporal Fusion Transformer* dan *Long Short-Term Memory* dengan

varian-variannya?

4. Fitur-fitur apa saja yang dapat membuat suatu model memiliki performa yang lebih baik serta apa hubungan antara fitur yang digunakan dengan performa model tersebut ?

I.3 Tujuan

Berdasarkan rumusan masalah yang sudah dijelaskan sebelumnya, berikut adalah tujuan dari penelitian ini.

1. Membuat model dengan arsitektur TFT dan varian LSTM pada data *time series* pasar saham Indonesia.
2. Mengevaluasi hasil performa dari model yang menggunakan arsitektur TFT dan varian LSTM pada data *time series* pasar saham Indonesia.
3. Menentukan fitur-fitur yang menjadi alasan mengapa salah satu model memiliki performa yang lebih baik daripada model lainnya pada data *time series* pasar saham Indonesia.

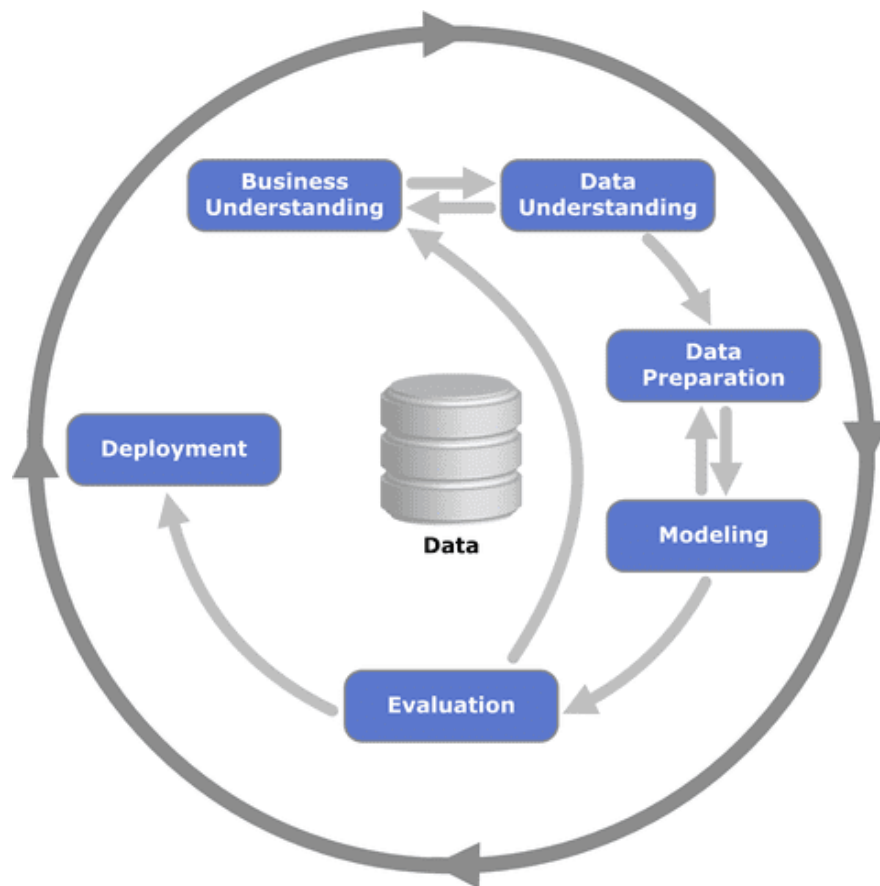
I.4 Batasan Masalah

Adapun beberapa batasan masalah yang diterapkan untuk penyusunan tugas akhir ini adalah sebagai berikut.

1. Dataset yang digunakan adalah data yang berasal dari Yahoo! Finance API dan juga data-data perusahaan yang terdaftar pada Bursa Efek Indonesia <https://www.idx.co.id/en/market-data/stocks-data/stock-list>
2. Bahasa pemrograman yang digunakan dalam melakukan pengembangan model dengan arsitektur TFT dan varian LSTM adalah Python 3.
3. Penelitian ini dilakukan menggunakan *device* Laptop Lenovo Legion 5 15IMH05 serta layanan *cloud computing* yaitu Google Colab.

I.5 Metodologi

Metodologi yang dipakai dalam penelitian kali ini adalah *Cross-Industry Process for Data Mining* (CRISP-DM). CRISP-DM merupakan salah satu metodologi umum yang digunakan dalam pengembangan proyek *data science* dikarenakan fase-fasenya yang fleksible untuk dilakukan sehingga cocok untuk digeneralisir kepada setiap domain-domain bisnis yang membutuhkan (Vorhies 2016). Metodologi ini terdiri atas 6 fase sesuai dengan gambar I.5.1.



Gambar I.5.1: Metodologi CRISP-DM. Sumber: (Vorhies 2016)

1. Business Understanding

Fase pertama dalam CRISP-DM adalah memahami tujuan bisnis yang diinginkan dan mengidentifikasi masalah bisnis yang ingin diselesaikan. Tujuan utama tahap ini adalah agar selarasnya solusi proyek *data science* dengan masalah bisnis yang ingin diselesaikan.

2. **Data Understanding**

Fase kedua dalam CRISP-DM adalah mengumpulkan data, memahami kualitas, struktur, serta relevansi, serta feasibilitas dari data yang akan digunakan untuk pengembangan proyek.

3. **Data Preparation**

Fase ketiga dalam CRISP-DM adalah melakukan pengolahan data untuk digunakan untuk fase selanjutnya.

4. **Modelling**

fase keempat dalam CRISP-DM adalah membangun model *machine learning* menggunakan algoritma yang sesuai dan data yang sudah dipersiapkan dari tahap sebelumnya.

5. **Evaluation**

Fase kelima dalam CRISP-DM adalah melakukan evaluasi performa dari model yang telah dibuat dari fase sebelumnya. Apabila model yang telah dibuat memiliki performa yang kurang memuaskan, maka iterasi dapat diulang kembali ke salah satu fase sebelumnya.

6. **Deployment**

Fase terakhir dalam CRISP-DM adalah melakukan *deployment* dari model yang kita gunakan sehingga model yang sudah dibuat dapat diakses atau digunakan.

BAB II

STUDI LITERATUR

II.1 Pasar Saham Indonesia

Gagasan mengenai pasar saham dapat ditelusuri berabad-abad yang lalu, bermula dari bentuk-bentuk awal perdagangan komoditas dan obligasi pemerintah, hingga menjadi platform canggih yang saat ini memfasilitasi pertukaran ekuitas perusahaan. Salah satu contoh awal dari sistem pasar yang relatif terorganisasi muncul pada abad ke-17 di Amsterdam dengan Perusahaan Hindia Timur Belanda (VOC), yang dianggap sebagai perusahaan publik pertama di dunia (Petram 2014; Neal 1990). Di sini, para investor dapat membeli dan menjual saham pada satu lokasi terpusat. Ini menjadi cikal bakal bursa saham modern (Goetzmann 2016). Seiring waktu, gagasan ini menyebar ke seluruh Eropa, dengan munculnya pasar di London dan Paris. Pada akhir abad ke-18 dan awal abad ke-19, Bursa Efek New York (NYSE) muncul di Amerika Serikat dan menjadi fondasi bagi jaringan global pasar keuangan yang saling terhubung (Markham 2002; Goetzmann 2016).

Ketika industrialisasi dan globalisasi meningkat pada abad ke-20, pasar saham tumbuh semakin cepat dan kompleks. Peralihan dari platform bursa yang fisik menjadi berbasis elektronik yang dapat beroperasi lintas zona waktu dan lintas negara memungkinkan partisipasi oleh yang lebih luas, baik oleh investor institusional maupun ritel, serta meningkatkan likuiditas dan efisiensi pasar. Liberalisasi aliran modal pada paruh kedua abad tersebut, ditambah dengan kemajuan teknologi, memungkinkan negara-negara berkembang membangun pasar saham-

nya sendiri. Hal ini menciptakan lanskap keuangan global yang lebih beragam dan saling terkait (OECD 2018).

Di Asia Tenggara, pasar saham memegang peran penting dalam menyalurkan modal domestik dan asing ke sektor-sektor kunci untuk mendorong pertumbuhan dan pembangunan ekonomi. Pada awalnya, pasar saham Indonesia beroperasi secara sederhana dengan jumlah perusahaan terbatas dan partisipasi investor yang minim (OJK 2007). Selama beberapa dekade, pasar ini mengalami transformasi signifikan. Pembentukan Bursa Efek Jakarta (BEJ) dan Bursa Efek Surabaya (BES) menyediakan gelanggang perdagangan saham yang terpisah, meskipun keduanya sempat menghadapi tantangan dalam mencapai kedalaman (*depth*) dan keluasan (*breadth*) dalam menjual emiten yang tercatat. Namun, reformasi regulasi, peningkatan tata kelola perusahaan, serta penguatan kerangka hukum membantu meningkatkan kepercayaan investor dan mendekatkan pasar modal Indonesia dengan standar global (OECD 2018).

Tahun 2007 menandai titik balik penting bagi pasar modal Indonesia, ketika Bursa Efek Jakarta dan Bursa Efek Surabaya bergabung membentuk Bursa Efek Indonesia (BEI). Konsolidasi ini bertujuan merampingkan operasional, mengurangi fragmentasi, dan memperkuat daya saing di tengah arus globalisasi (OJK 2007). Bersamaan dengan reorganisasi institusional, regulator keuangan seperti Otoritas Jasa Keuangan (OJK) menerapkan langkah-langkah untuk meningkatkan integritas pasar, transparansi, dan perlindungan investor. Upaya ini sejalan dengan praktik terbaik internasional guna mencerminkan kesadaran bahwa pasar saham yang berfungsi dengan baik sangat penting untuk menarik investasi jangka panjang, mendukung pembiayaan perusahaan, dan pada akhirnya berkontribusi pada stabilitas serta pertumbuhan ekonomi.

Bukti empiris menunjukkan bahwa peran pasar saham Indonesia terus berkembang, sejalan dengan peningkatan jumlah perusahaan yang tercatat dan diversifikasi sektor usaha—dari pertanian, barang konsumsi, keuangan, infrastruktur, hingga perusahaan berbasis teknologi. Diversifikasi ini meningkatkan ketahanan pasar, mengurangi *concentration risk*, dan mendorong investor untuk melihat lanskap korporasi Indonesia secara lebih cermat. Masuknya investor asing, yang

didukung oleh lingkungan regulasi yang lebih kondusif, turut menambah likuiditas. Investor lokal juga diuntungkan oleh inisiatif peningkatan literasi keuangan serta platform perdagangan yang lebih mudah digunakan (OECD 2018).

Perkembangan lain yang signifikan adalah berubahnya profil investor di Indonesia. Jika pada masa lalu pasar didominasi oleh institusi besar dan individu borjuis, kini partisipasi investor ritel semakin meningkat. Kampanye pemerintah, program edukasi, serta layanan pialang daring telah membuka akses perdagangan saham untuk segmen yang sebelumnya tidak dapat dijangkau dalam masyarakat. Demokratisasi akses ini membentuk budaya investasi yang lebih inklusif dan dapat menstabilkan pasar dengan memperluas basis atau fondasi dari modal-modal jangka panjang. Dengan demikian, pasar saham Indonesia kini tidak lagi menjadi domain eksklusif kelompok tertentu, melainkan menjadi platform tempat masyarakat umum dapat turut andil dalam perjalanan ekonomi nasional.

Meski begitu, pasar saham Indonesia tetap rentan terhadap guncangan eksternal dan volatilitas regional, yang merupakan tantangan yang biasa dihadapi oleh banyak pasar negara berkembang (Torre and Schmukler 2007). Faktor seperti fluktuasi harga komoditas global, ketegangan geopolitik, serta perubahan kebijakan moneter internasional semuanya dapat memengaruhi sentimen investor dan aliran modal (Bloom et al. 2019). Faktor domestik—termasuk perubahan kebijakan pemerintah, indikator makroekonomi, dan standar tata kelola perusahaan—juga berinteraksi membentuk dinamika pasar. Walaupun pasar saham Indonesia telah matang dan terintegrasi dengan keuangan global, ia tetap beroperasi dalam ekosistem kompleks yang memerlukan pengelolaan risiko yang berkelanjutan (IMF, 2022).

Upaya untuk meningkatkan infrastruktur pasar, menerapkan kerangka regulasi yang kokoh, serta mengadopsi inovasi teknologi yang lebih canggih akan semakin memperkuat posisi Indonesia sebagai tujuan investasi utama di Asia Tenggara. Kemungkinan pengenalan instrumen keuangan yang lebih beragam, integrasi yang lebih dalam dengan pasar modal internasional, serta penerapan teknologi digital mutakhir seperti perdagangan algoritmik dapat membawa era baru efisiensi dan transparansi. Di saat yang sama, faktor Lingkungan, Sosial, dan Tata

Kelola (ESG) kian menonjol, sejalan dengan tren investasi berkelanjutan di tingkat global. Hal ini yang mendorong perusahaan-perusahaan untuk memenuhi standar-standar yang lebih tinggi serta tanggung jawab korporasi, yang berpotensi menarik kelompok investor yang lebih sadar dan bertanggung jawab akan ESG.

II.2 Kecerdasan Buatan

Kecerdasan Buatan (*Artificial Intelligence* atau AI) adalah cabang ilmu komputer yang bertujuan menciptakan mesin serta sistem yang mampu menjalankan tugas-tugas yang biasanya memerlukan kecerdasan manusia (Russell and Norvig 2020). Bidang ini berupaya membuat komputer dapat memahami, belajar dari lingkungan, dan berinteraksi dengan cara yang menyerupai atau bahkan melampaui kemampuan kognitif manusia dalam mengenali pola, mengambil keputusan, dan memecahkan masalah tanpa kita sebagai manusia perlu memberitahu mesin secara eksak apa yang perlu mereka lakukan.

Sejak kemunculannya sebagai disiplin akademik pada pertengahan abad ke-20, AI telah dipengaruhi oleh berbagai bidang ilmu, seperti matematika, psikologi, linguistik, biologi, filsafat, serta rekayasa komputer. Pendekatan awal riset AI banyak berfondasi pada *symbollic logic* dan aturan-aturan eksplisit yang meniru proses berpikir manusia (Russell and Norvig 2020). Namun, seiring waktu, AI berkembang dengan memasukkan metode statistik dan pendekatan berbasis data, termasuk pembelajaran mesin (*machine learning*). Pembelajaran mesin memungkinkan sistem meningkatkan kinerjanya melalui pengalaman secara iteratif (Goodfellow, Bengio, and Courville 2016). Salah satu cabang pembelajaran mesin yang menonjol adalah pembelajaran mendalam (*deep learning*), yang memanfaatkan jaringan saraf buatan yang terinspirasi oleh struktur dan fungsi otak manusia, untuk mengenali pola dalam data yang kompleks.

Saat ini, AI telah diterapkan pada berbagai aspek kehidupan sehari-hari, mulai dari asisten virtual berbasis suara, kendaraan otonom, sistem rekomendasi, hingga analisis data tingkat lanjut pada bidang-bidang sains lainnya. Dalam

sektor-sektor seperti kesehatan, keuangan, manufaktur, dan pendidikan, AI dapat meningkatkan efisiensi, produktivitas, serta kualitas pengambilan keputusan. Meskipun perkembangan AI melaju pesat, isu tentang etika, transparansi, dan akuntabilitas tetap harus diprioritaskan untuk menjadi perhatian penting untuk mendorong adanya diskusi mengenai pengembangan dan penerapan AI secara bertanggung jawab agar tidak merugikan umat manusia (Jobin, Ienca, and Vayena 2019).

II.3 Pembelajaran Mesin

Pembelajaran mesin (*machine learning*) adalah salah satu cabang dari kecerdasan buatan yang berfokus pada pengembangan algoritma dan model yang dapat belajar dari data tanpa secara eksplisit diprogram ulang untuk setiap tugas (Mitchell 1997). Dalam pembelajaran mesin, komputer “belajar” dengan menganalisis data dengan jumlah yang besar untuk menemukan pola atau hubungan yang kemudian dapat digunakan untuk membuat prediksi atau mengambil keputusan di masa mendatang. Pendekatan ini berbeda dari pemrograman tradisional, di mana kita menuliskan aturan secara eksplisit di dalam kode, sedangkan pada pembelajaran mesin, sistem secara otomatis menyimpulkan aturan ataupun cara untuk melakukan sebuah *task* tersebut dari data.

Secara umum, pembelajaran mesin dapat dibagi menjadi tiga kategori utama: *supervised learning*, *unsupervised learning*, dan *reinforcement learning* (Goodfellow, Bengio, and Courville 2016). *Supervised learning* membangun model menggunakan data yang telah dilabeli untuk “melatih” model agar dapat memprediksi label pada data baru yang belum pernah dilihat oleh model tersebut. *Unsupervised learning* tidak memerlukan label karena berfokus pada eksplorasi struktur atau pola tersembunyi di dalam data tersebut. Sementara itu, *reinforcement learning* menggunakan interaksi sistem dengan *environment*, di mana model secara iteratif memperbarui strateginya dalam memenuhi atau melaksanakan *task* berdasarkan umpan balik yang dapat berupa “reward” atau “punishment”.

Perkembangan pembelajaran mesin sangat pesat berkat ketersediaan data yang

semakin melimpah dan peningkatan daya komputasi yang besar. Metode ini telah dimanfaatkan dalam berbagai bidang, seperti pengenalan suara dan wajah, klasifikasi dokumen, *fraud detection*, analisis pasar saham, serta diagnosis medis berbasis citra (Mohri, Rostamizadeh, and Talwalkar 2018). Seiring dengan kemajuan pembelajaran mesin, tantangan dalam pembelajaran mesin juga muncul. Salah satu contohnya adalah dalam hal interpretabilitas model, privasi data, dan bias algoritma. Oleh karena itu, penelitian dan inovasi dalam pembelajaran mesin terus dilakukan untuk memastikan agar teknologi ini tetap dapat digunakan, aman, serta bermanfaat secara luas bagi masyarakat.

II.4 Time Series data

Data deret waktu (*time series data*) adalah jenis data yang direkam, dikumpulkan, atau diamati secara berurutan berdasarkan dimensi waktu (Chatfield 2003). Karakteristik utama dari data deret waktu adalah adanya ketergantungan atau keterkaitan antara nilai pada suatu point waktu dengan nilai-nilai pada point-point sebelumnya. Ketergantungan ini membuat pendekatan analisis data deret waktu berbeda dengan analisis data yang tidak mengkonsiderasi urutan data.

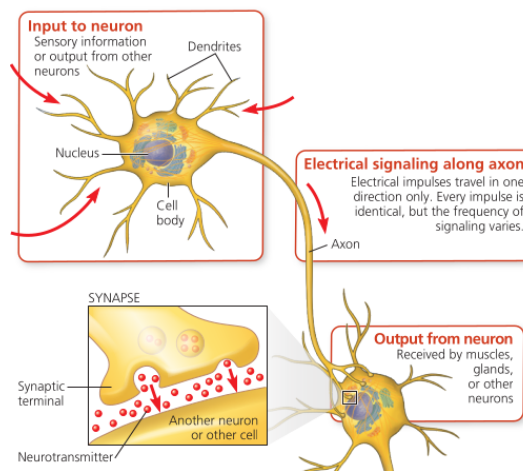
Terdapat banyak jenis data yang disimpan dalam tipe *time series*. Beberapa dari contohnya adalah harga saham harian dari sebuah perusahaan, jumlah penumpang kereta setiap bulan, tingkat inflasi tahunan, atau konsumsi energi listrik per jam. Pola yang muncul dalam data deret waktu seringkali mencerminkan proses atau fenomena mendasar yang terjadi secara dinamis, misalnya tren meningkat dan menurun ataupun pola *seasonality*. Memahami dan mengetahui eksistensi dari pola-pola ini dapat membantu dalam membuat keputusan, perencanaan strategis, serta prediksi masa depan yang lebih tepat (Hyndman and Athanasopoulos 2021).

Dalam analisis data deret waktu, berbagai metode statistik dan algoritma pembelajaran mesin dapat digunakan. Metode tradisional seperti *Autoregressive Integrated Moving Average* (ARIMA) atau model *Vector Autoregression* (VAR) telah lama diaplikasikan untuk memodelkan dan memprediksi data deret wak-

tu, namun seiring meningkatnya kompleksitas data serta kebutuhan akan model yang lebih fleksibel, pendekatan modern seperti model jaringan saraf dan arsitektur Transformer mulai banyak diterapkan (Goodfellow, Bengio, and Courville 2016). Perkembangan ini terjadi karena model-model lama seperti ARIMA maupun VAR kurang mampu mempertimbangkan efek nilai masa lalu yang sangat jauh terhadap nilai masa kini, terutama dalam interval waktu yang panjang, sehingga diperlukan model yang dapat mempelajari pola non-linear dan hubungan dependensi jangka panjang.

II.5 Neural Network

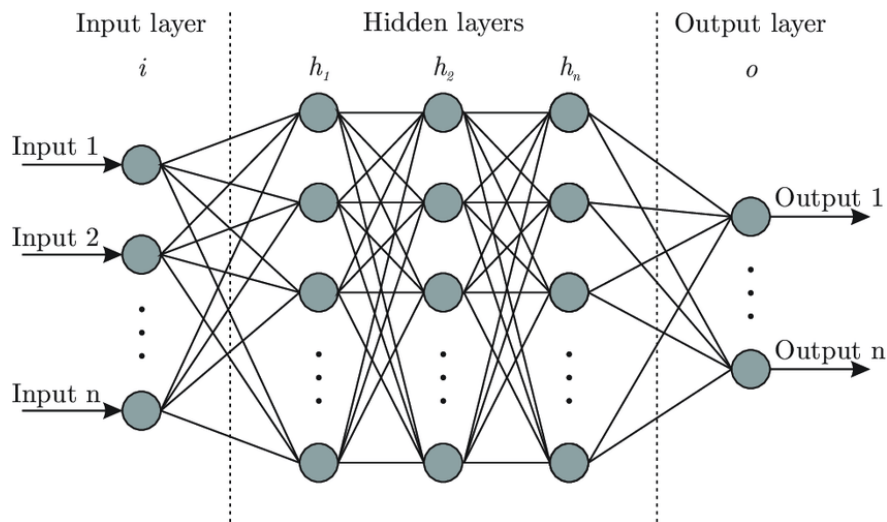
Dikembangkannya *neural network* sebagai salah satu solusi untuk membuat *artificial intelligence* terinspirasi dari munculnya ide bahwa kecerdasan manusia serta struktur otak manusia tidak memiliki karakteristik yang sama seperti komputer (need citation). Otak manusia tidak bekerja secara *rigid* seperti komputer. Otak manusia dapat bekerja secara adaptif apabila terdapat informasi baru yang datang. Hal ini disebabkan oleh sifat *neuroplasticity* dari otak (need citation).



Gambar II.5.1: *Biological neuron*(Orr et al. 2021)

Pada gambar II.5.1, informasi didapatkan melalui *dendrites*. Setelah itu, informasi akan diolah oleh *nucleus* lalu akan diteruskan oleh *axon*. *Axon* akan menentukan seberapa besar signal yang sudah diolah oleh *nucleus* akan diteruskan ke neuron - neuron lainnya.

Konsep neuron tersebut dicoba ditiru oleh ilmuwan - ilmuwan ilmu komputer. Kecerdasan buatan dapat dimodelkan sebagai sebuah grap dengan *artificial neural network* yang terdiri dari *input nodes*, *computing nodes*, dan *output nodes*. Fungsi dari *input nodes* adalah hanya sebagai *node* yang akan melakukan propagasi informasi dari *node* itu sendiri menuju ke *computing nodes*. Pada *computing node*, akan terjadi komputasi. Setelah itu, informasi akan dipropagasikan lagi sampai menuju *output nodes*. Ketiga tipe *node* tersebut direpresentasikan di gambar II.5.2 dimana *input nodes* terdapat pada *input layer*, *computing nodes* terdapat pada *hidden layer*, dan *output nodes* terdapat pada *output layer*.



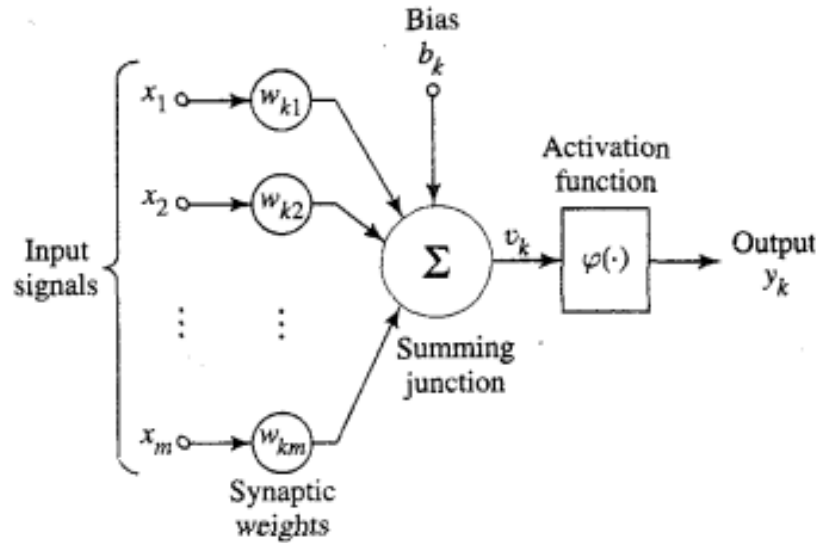
Gambar II.5.2: Graf *Neural Network*. (Waskiewicz 2021)

Fungsi dari *input nodes* dan *output nodes* sudah *self-explanatory* dimana kedua jenis *node* tersebut hanya untuk sebagai pempropagasi informasi dan penyimpanan hasil kalkulasi *computing node*. *Computing node* atau yang sering juga disebut dengan *perceptron* memiliki struktur yang berbeda. Pada gambar II.5.3, *computing node* berindeks K dapat direpresentasikan sebagai sebuah neuron yang menerima M buah *input* dari dendritnya. Pertama-tama, akan datang M buah sinyal melalui dendrit dari neuron tersebut. Setiap dendrit pada *computing node* memiliki *weight* yang merupakan rasio dari seberapa besar informasi dari dendrit tersebut berkontribusi kepada neuron tersebut yang dapat disimbolkan sebagai $W_{k,i}$. Perlu juga ditambahkan sebuah variabel *bias* yang agnostik terhadap hasil perhitungan pada *layer sebelumnya*. Variabel ini dapat kita simbolkan sebagai

$$\vec{v}_k = \sum_{i=0}^M w_i x_i \quad (\text{II.1})$$

Persamaan II.1: *Summing Function*

sinyal yang datang dari dendrit dengan indeks ke 0 dengan *weight* 1. Semua sinyal tersebut akan dimasukkan ke dalam *summing function* pada II.1.



Gambar II.5.3: Perceptron buatan (Haykin 1998)

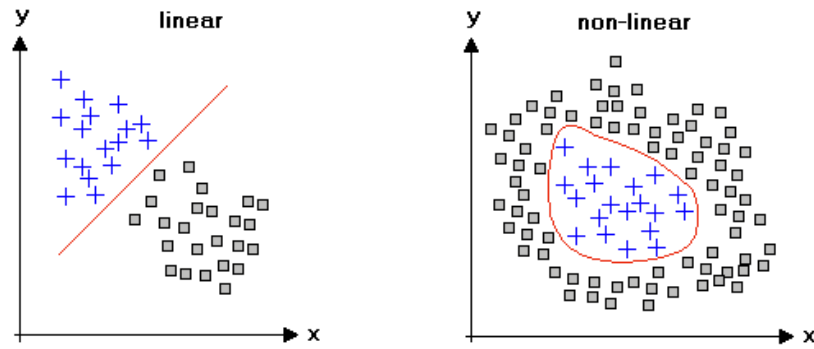
Nilai yang telah didapatkan dari *summing function* akan lanjut diproses oleh *activation function*. *Activation function* disimbolkan sebagai $|\phi(\cdot)|$ pada gambar II.5.3. *Activation function* akan menerima input lalu akan mengubahnya menjadi nilai pada *range* $[0, N]$. *Activation function* diperlukan guna untuk mengatasi ketidaklinearitasan data seperti pada gambar II.5.4. Apabila hasil *summing function* tidak diproses lebih lanjut menggunakan *activation function*, maka semua hasil komputasi hanyalah sebuah proses *matrix multiplication* yang tidak dapat mengatasi ketidaklinearitasan data. Salah satu *activation function* yang paling banyak digunakan adalah ReLU (*Rectified Linear Unit*) *function* yang ada pada persamaan II.2. ReLU banyak digunakan sebagai *activation function* dikarenakan ReLU tidak akan mengalami *vanishing gradient* ketika model melakukan *backward pass* dalam *training weights* dari dendrit setiap neuron.

$$\text{ReLU}(x) = \begin{cases} x, & \text{if } x > 0 \\ 0, & \text{if } x \leq 0 \end{cases} \quad (\text{II.2})$$

Persamaan II.2: *Rectified Linear Unit Function*

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (\text{II.3})$$

Persamaan II.3: *Mean Squared Error*



Gambar II.5.4: Data non-linear (Learning 2021)

Untuk setiap iterasi *pass neural network*, akan dikalkulasikan sebuah nilai menggunakan *loss function*. Semakin tinggi nilai dari *loss function*, maka semakin buruk model tersebut bekerja. Terdapat banyak sekali tipe dari *loss function*. Salah satunya adalah *Mean Squared Error* seperti pada persamaan II.3. *Loss Function* akan mengambil nilai pada *layer* terakhir atau *output layer* lalu melakukan komparasi untuk mencari tahu seberapa jauh model *drifted* dari prediksinya.

Secara umum, satu iterasi dalam *training neural network* terdiri atas 2 tahap yaitu *forward pass* dan *backward pass*. *Forward pass* akan melakukan propagasi informasi dari *input layer* menuju ke *output layer* dimana akan terjadi komputasi pada setiap neuron yang dilewati. Setelah itu, akan dilakukan komputasi performa model menggunakan *loss function*. *Loss function* ini nantinya akan digunakan pada saat *backward pass* dimana *weight* dari setiap dendrit di neuron akan dikalkulasi ulang respektif terhadap nilai dari *loss function*. *Training* dari

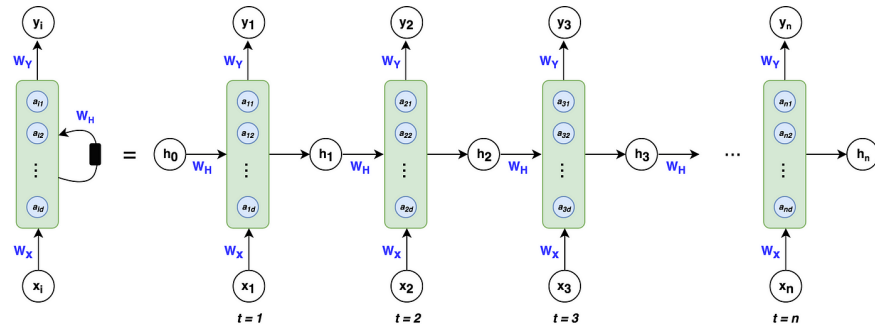
model *neural network* terjadi pada saat *backward pass* ini.

II.5.1 Recurrent Neural Network

RNN merupakan jenis *neural network* yang dirancang untuk memproses data sekuensial dengan mempertahankan informasi dari waktu sebelumnya. *Vanilla neural network* tidak peduli dengan urutan sekuens data sehingga semua data itu dikonsiderasi sebagai sama.

II.5.1.1 Vanilla RNN

Recurrent Neural Network (RNN) secara umum adalah jenis *neural network* yang dirancang untuk memproses data secara sekuensial. Pada RNN, informasi tidak hanya mengalir dari lapisan *input* ke lapisan *output*, tetapi juga “diputar kembali” (*recurred*) ke dalam *hidden state* yang memungkinkan jaringan menyimpan konteks dari waktu ke waktu. Arsitektur RNN dapat dilihat pada gambar II.5.5



Gambar II.5.5: Arsitektur RNN (S November 2024)

Ketika menangani data sekuensial (data teks, urutan gambar, sinyal suara, atau *time series*), urutan (*order*) data sangatlah penting. Contohnya, dalam pemrosesan teks, kata di posisi ke-5 memiliki konteks makna yang ditentukan oleh kata-kata sebelumnya (posisi 1–4). *Vanilla RNN* adalah bentuk paling sederhana dari RNN yang menerapkan ide “menyimpan konteks” tersebut melalui sebuah *hidden state* yang diperbarui di setiap langkah waktu.

Arsitektur *Vanilla RNN* dapat direpresentasikan sebagai berikut.

$$\mathbf{h}_t = \phi(W_{xh} \mathbf{x}_t + W_{hh} \mathbf{h}_{t-1} + \mathbf{b}_h) \quad (\text{II.4})$$

Persamaan II.4: *Hidden state* pada *Vanilla RNN*

$$\mathbf{y}_t = W_{hy} \mathbf{h}_t + \mathbf{b}_y \quad (\text{II.5})$$

Persamaan II.5: *Output* pada *Vanilla RNN*

1. **Input** (x_t): Vektor yang merepresentasikan data pada waktu t . Sebagai contoh, representasi kata (*word embedding*) dalam pengolahan teks, atau nilai sesuatu pada waktu t dalam *time series*.
2. **Hidden state** (H_t): “Memori internal” RNN yang menampung informasi dari langkah waktu sebelumnya (H_{t-1}).
3. **Output** (y_t): Vektor yang menjadi keluaran pada waktu T . Dalam beberapa kasus, *Neural network* hanya mengeluarkan hasil di langkah terakhir. Akan tetapi pada kasus lain (misalnya *language modeling*), *neural network* bisa mengeluarkan hasil di setiap langkah.

Secara matematis, proses ini dapat direpresentasikan oleh dua persamaan utama yaitu persamaan II.4 dan II.5 dengan keterangan :

- $\mathbf{x}_t \in \mathbb{R}^d$: *Input* pada waktu t .
- $\mathbf{h}_{t-1} \in \mathbb{R}^h$: *Hidden state* pada waktu $t - 1$.
- $\mathbf{W}_{xh} \in \mathbb{R}^{h \times d}$: Bobot transformasi dari *input* ke *hidden state*.
- $\mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$: Bobot transformasi dari *hidden state* sebelumnya ke *hidden state* baru.
- $\mathbf{b}_h \in \mathbb{R}^h$: *Bias* untuk *hidden state*.
- $\mathbf{W}_{hy} \in \mathbb{R}^{o \times h}$: Bobot transformasi dari *hidden state* ke *output*.
- $\mathbf{b}_y \in \mathbb{R}^o$: *Bias* untuk *output*.
- $\phi(\cdot)$: *Activation function*.

Forward pass dalam RNN adalah proses menghitung h_t secara berurutan dari $t = 1$ sampai $t = N$. Proses *forward pass* dapat dimodelkan sebagai berikut:

1. Inisialisasi $\mathbf{h}_0$, umumnya sebagai vektor nol ($\mathbf{0}$).
2. Untuk setiap waktu t dalam $\{1, 2, \dots, N\}$:
 - (a) Hitung $\mathbf{h}_t = \phi(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$.
 - (b) Hitung $\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$.

Setelah mendapatkan *output* di setiap T , *neural network* akan menghitung *loss function* yang mengukur seberapa baik prediksi $\mathbf{y}_t$ dibandingkan dengan nilai sebenarnya $\hat{\mathbf{y}}_t$. *Backward Pass* dan *Backpropagation Through Time (BPTT)* Untuk melakukan *training* pada RNN. Akan digunakan *Backpropagation Through Time (BPTT)* yang merupakan penerapan *chain rule* secara berurutan dari waktu $t = N$ “kembali” ke $t = 1$. Pertama, akan dilakukan perhitungan turunan $\frac{\partial \text{Loss}}{\partial \mathbf{h}_t}$ pada setiap t . Selanjutnya, pada setiap langkah waktu, turunan akan dicari menggunakan *chain rule* ($\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}$, $\frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{xh}}$, $\frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}}$, dan $\frac{\partial \text{Loss}}{\partial \mathbf{y}_t}$ untuk menemukan gradien parameter ($\frac{\partial \text{Loss}}{\partial \mathbf{W}_{xh}}$, $\frac{\partial \text{Loss}}{\partial \mathbf{W}_{hh}}$, $\dots$). *Weight* dari setiap neuron ($\mathbf{W}_{xh}$, $\mathbf{W}_{hh}$, $\mathbf{W}_{hy}$, $\mathbf{b}_h$, $\mathbf{b}_y$) akan diperbarui menggunakan algoritma optimisasi seperti *Stochastic Gradient Descent (SGD)*, *Momentum*, atau *Adam*. Seluruh iterasi ini akan diulang sampai nilai dari *loss function* sudah tercapai atau sudah dapat diterima.

Terdapat beberapa masalah yang dapat muncul pada RNN. Ketika urutan sekuens terlalu panjang, gradien yang dihitung lewat BPTT dapat mengecil (≈ 0) secara eksponensial sehingga *weight* di langkah-langkah waktu awal hampir tidak diperbarui. Hal tersebut disebut sebagai *vanishing gradient*. Terdapat pula masalah lain yang bernama *exploding gradient*. Berlawanan dengan *vanishing gradient*, gradien dapat tumbuh sangat besar ($\gg 1$), mengakibatkan pembaruan *weight* yang ekstrem dan tidak stabil.

Vanilla RNN tidak efektif dalam hal komputasi karena prosesnya tidak dapat berjalan secara paralel. Setiap langkah waktu harus diolah satu per satu sehingga kecepatan pemrosesan menjadi relatif lambat jika dibandingkan dengan metode lain yang mendukung komputasi paralel. Kondisi ini dapat memperpanjang du-

rasi *trainig* seiring dengan meningkatnya ukuran dari *dataset* serta *time-length* dari data yang digunakan.

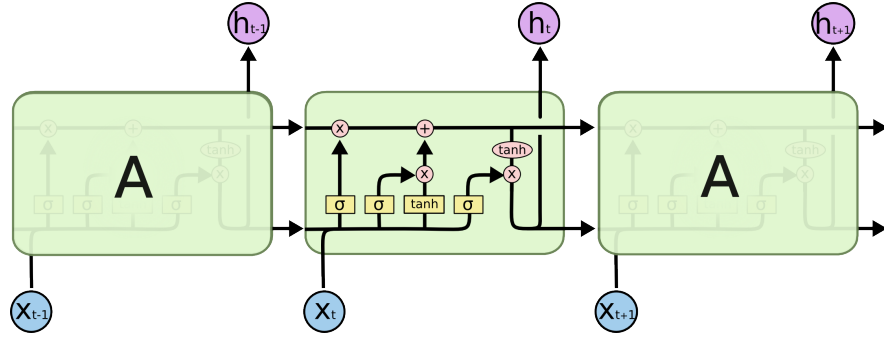
II.5.1.2 Gated RNN

Salah satu solusi untuk mengatasi permasalahan *vanishing* dan *exploding gradient* pada *Vanilla RNN* adalah dengan menambahkan mekanisme *gating* ke dalam arsitektur RNN. Arsitektur RNN dengan mekanisme *gating* ini sering disebut sebagai *Gated RNN*. Dua jenis Gated RNN yang paling populer adalah *Long Short-Term Memory (LSTM)* dan *Gated Recurrent Unit (GRU)*.

Ide utama dari Gated RNN adalah menambahkan “gerbang-gerbang” (*gates*) yang secara eksplisit mengatur aliran informasi di dalam *hidden state* sehingga model dapat melakukan *retaining* informasi penting lebih pada *timepoint* yang lebih lawas dan membuang informasi yang dianggap tidak relevan. Pada LSTM, terdapat 3 jenis gerbang yaitu *forget gate*, *input gate* dan *output gate*. Pada GRU, beberapa gerbang di LSTM digabungkan sehingga memiliki arsitektur yang lebih simpel.

1. LSTM (*Long Short-Term Memory*)

LSTM dikemukakan pertama kali oleh (Hochreiter and Schmidhuber 1997) *long-term dependencies* yang dialami oleh *Vanilla RNN*. LSTM menambahkan beberapa komponen *gates* dan sebuah *cell state* (C_t) yang berfungsi sebagai “jalur informasi” jangka panjang. Tujuannya adalah untuk mencegah informasi penting mengalami *vanishing gradient* ketika dilatih dengan *Backpropagation Through Time (BPTT)* pada sekuens yang panjang. Arsitektur tersebut dapat terlihat pada gambar II.5.6.



Gambar II.5.6: Diagram LSTM (Olah 2015)

LSTM mengatur aliran informasi melalui tiga jenis gerbang utama (*forget gate*, *input gate*, *output gate*) dan satu *cell state*. Pada setiap langkah waktu t , kita akan menyimpan :

- $\mathbf{x}_t$: *input* pada waktu t .
- $\mathbf{h}_{t-1}$: *hidden state* pada waktu $t - 1$.
- $\mathbf{C}_{t-1}$: *cell state* pada waktu $t - 1$.

Untuk setiap langkah waktu di setiap iterasi, kita akan melakukan komputasi untuk setiap gerbang ini.

1. Forget Gate ($\mathbf{f}_t$) Gerbang ini memutuskan seberapa banyak memori dalam $\mathbf{C}_{t-1}$ yang perlu dibuang.

$$\mathbf{f}_t = \sigma\left(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f\right)$$

$\sigma(\cdot)$ adalah *sigmoid function* yang memetakan nilai *input* ke rentang $[0, 1]$. Nilai 1 berarti tetap mempertahankan memori dan 0 berarti buang sepenuhnya.

2. Input Gate ($\mathbf{i}_t$) & Candidate Cell State ($\tilde{\mathbf{C}}_t$)

- (a) $\mathbf{i}_t$ (*input gate*) menentukan seberapa besar memori baru yang akan disimpan ke dalam $\mathbf{C}_t$.

$$\mathbf{i}_t = \sigma\left(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i\right)$$

- (b) $\tilde{\mathbf{C}}_t$ (*candidate cell state*) adalah kandidat memori baru yang akan ditambahkan. Fungsi tanh akan digunakan agar nilai yang dihasilkan berada di rentang $[-1, 1]$.

$$\tilde{\mathbf{C}}_t = \tanh\left(W_C[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C\right)$$

Kedua komponen ini bersama-sama menentukan informasi apa yang akan ditambahkan ke $\mathbf{C}_t$.

3. Update Cell State ($\mathbf{C}_t$) Setelah $\mathbf{f}_t$ memutuskan seberapa banyak informasi lama dipertahankan, dan $\mathbf{i}_t$ menentukan seberapa banyak memori baru (melalui $\tilde{\mathbf{C}}_t$) yang ditambahkan, $\mathbf{C}_t$ diperbarui dengan:

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$$

Simbol $\odot$ menyatakan perkalian *element-wise* (perkalian *one-on-one* antar komponen vektor). Proses ini memungkinkan LSTM mempertahankan sebagian memori dari $\mathbf{C}_{t-1}$, sekaligus menambahkan informasi baru dari $\tilde{\mathbf{C}}_t$.

4. Output Gate ($\mathbf{o}_t$) & Hidden State ($\mathbf{h}_t$)

- (a) **Output Gate ($\mathbf{o}_t$)** memutuskan seberapa banyak informasi (yang tersimpan di $\mathbf{C}_t$) akan dipropagasikan ke $\mathbf{h}_t$.

$$\mathbf{o}_t = \sigma\left(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o\right)$$

- (b) **Hidden State ($\mathbf{h}_t$)** kemudian dihitung dengan memfilter $\mathbf{C}_t$ melalui tanh, dan kemudian dikalikan dengan $\mathbf{o}_t$:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t)$$

Berikut adalah penjelasan detail dari setiap *cell* dan *gate*.

- (a) **Cell State ($\mathbf{C}_t$)**: Aliran informasi jangka panjang yang mengalir di sepanjang urutan waktu. LSTM memungkinkan informasi mengalir tanpa banyak modifikasi dengan mekanisme gerbang.

- (b) **Forget Gate** ($\mathbf{f}_t$): Menentukan informasi mana yang akan dilupakan dari $\mathbf{C}_{t-1}$.

$$\mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

σ adalah *sigmoid function* yang *output*-nya berada di $[0, 1]$. Nilai 0 berarti buang sedangkan 1 berarti pertahankan.

- (c) **Input Gate** ($\mathbf{i}_t$): Mengatur seberapa besar informasi baru yang akan disimpan ke dalam $\mathbf{C}_t$.

$$\mathbf{i}_t = \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

- (d) **Candidate Cell State** ($\tilde{\mathbf{C}}_t$): Kandidat memori baru yang akan ditambahkan ke dalam $\mathbf{C}_t$.

$$\tilde{\mathbf{C}}_t = \tanh(W_C[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C)$$

- (e) **Update Cell State**: Setelah memutuskan seberapa banyak informasi yang dibuang dan seberapa banyak yang ditambahkan $\mathbf{C}_t$ diupdate dengan:

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$$

Di sini, $\odot$ menandakan operasi perkalian *elementwise*.

- (f) **Output Gate** ($\mathbf{o}_t$): Mengatur seberapa banyak informasi pada $\mathbf{C}_t$ yang diambil untuk menghasilkan $\mathbf{h}_t$.

$$\mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

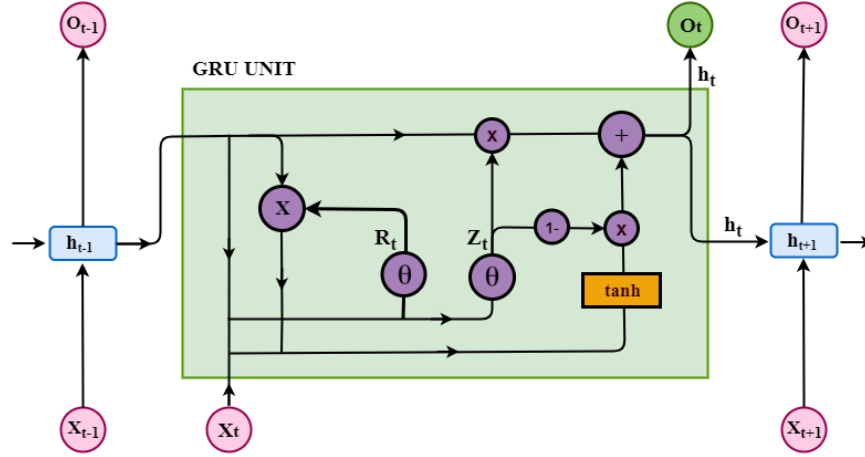
- (g) **Hidden State** ($\mathbf{h}_t$): Nilai $\mathbf{h}_t$ dihitung dengan “menyaring” $\mathbf{C}_t$ melalui $\tanh$ dan dikalikan dengan $\mathbf{o}_t$:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t)$$

2. GRU (*Gated Recurrent Unit*)

Gated Recurrent Unit (GRU) pertama kali diperkenalkan oleh Cho et al. (2014) dan dapat dianggap sebagai variasi *gating* yang lebih sederhana dari LSTM.

GRU menggabungkan beberapa gerbang LSTM menjadi satu sehingga arsitekturnya lebih ringkas dan memiliki jumlah parameter yang lebih sedikit dibandingkan LSTM. Meskipun demikian, GRU tetap mampu mengatasi masalah *vanishing gradient* dan *exploding gradient* dengan lebih baik daripada *Vanilla RNN*. Arsitektur dari GRU dapat dilihat pada gambar II.5.7.



Gambar II.5.7: Diagram Sel GRU (Bibi et al. 2020)

Pada GRU, kita memiliki dua gerbang utama, yaitu **update gate** (z_t) dan **reset gate** (r_t). Secara umum, komputasi pada waktu t dapat dijabarkan sebagai berikut:

(a) **Update Gate** (z_t)

Gerbang ini berfungsi mengontrol seberapa besar memori lama (h_{t-1}) akan tetap dipertahankan dan seberapa besar memori baru akan ditambahkan:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

Nilai $\sigma(\cdot)$ dalam rentang $[0, 1]$. Semakin tinggi z_t , semakin besar kontribusi memori baru ($\tilde{h}_t$) terhadap pembentukan h_t .

(b) **Reset Gate** (r_t)

Gerbang ini menentukan seberapa besar memori lama (h_{t-1}) akan dibang sebelum dikombinasikan dengan *input* baru (x_t):

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

Jika $\mathbf{r}_t$ bernilai mendekati 0, maka mayoritas memori lama akan dilupakan. Jika bernilai mendekati 1, maka mayoritas memori lama akan banyak dipertahankan.

(c) ***Candidate Hidden State*** ($\tilde{\mathbf{h}}_t$)

Setelah diperoleh $\mathbf{r}_t$, model akan menghitung *candidate hidden state* $\tilde{\mathbf{h}}_t$ dengan:

$$\tilde{\mathbf{h}}_t = \tanh\left(W_h[(\mathbf{r}_t \odot \mathbf{h}_{t-1}), \mathbf{x}_t] + \mathbf{b}_h\right)$$

Di sini, $\mathbf{r}_t \odot \mathbf{h}_{t-1}$ menunjukkan bahwa memori lama $\mathbf{h}_{t-1}$ terlebih dahulu difilter oleh $\mathbf{r}_t$ sebelum dicampurkan dengan *input* baru $\mathbf{x}_t$.

(d) ***Hidden State*** ($\mathbf{h}_t$)

Nilai $\mathbf{h}_t$ diperbarui dengan memadukan $\mathbf{h}_{t-1}$ dan $\tilde{\mathbf{h}}_t$ sesuai seberapa besar $\mathbf{z}_t$ memperbolehkan memori baru untuk menggantikan memori lama:

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

Apabila $\mathbf{z}_t$ bernilai besar (mendekati 1), lebih banyak informasi baru $\tilde{\mathbf{h}}_t$ yang akan berkontribusi. Sebaliknya, jika $\mathbf{z}_t$ bernilai kecil (mendekati 0), memori lama $\mathbf{h}_{t-1}$ akan lebih dominan dipertahankan.

GRU memiliki arsitektur yang lebih sederhana dibandingkan LSTM karena hanya menggunakan dua gerbang, yaitu *update* dan *reset*, sehingga jumlah parameternya lebih sedikit. Hal ini membuat waktu *training* dan memori yang dibutuhkan menjadi lebih efisien tanpa mengurangi kemampuan GRU untuk mengatasi masalah *vanishing gradient*. GRU tidak memiliki *cell state* terpisah dan hanya mengandalkan *hidden state* Tunggal sehingga dalam beberapa konteks tertentu, LSTM dengan *cell state* khusus dapat menyimpan informasi lebih stabil. Selain itu, GRU, seperti halnya LSTM, masih memproses data secara berurutan sehingga belum mampu menyaingi model berbasis *attention* (misalnya *Transformers*) dalam hal paralelisasi. Performa GRU juga sangat bergantung pada konfigurasi *hyperparameter* (*learning rate*, jumlah *hidden units*, dan sebagainya).

II.5.1.3 Varian RNN lainnya

Selain LSTM dan GRU, telah dikembangkan juga beberapa varian lainnya:

1. **Bi-directional RNN**

Melakukan *processing input* dari dua arah (depan-belakang).

2. **Stacked RNN**

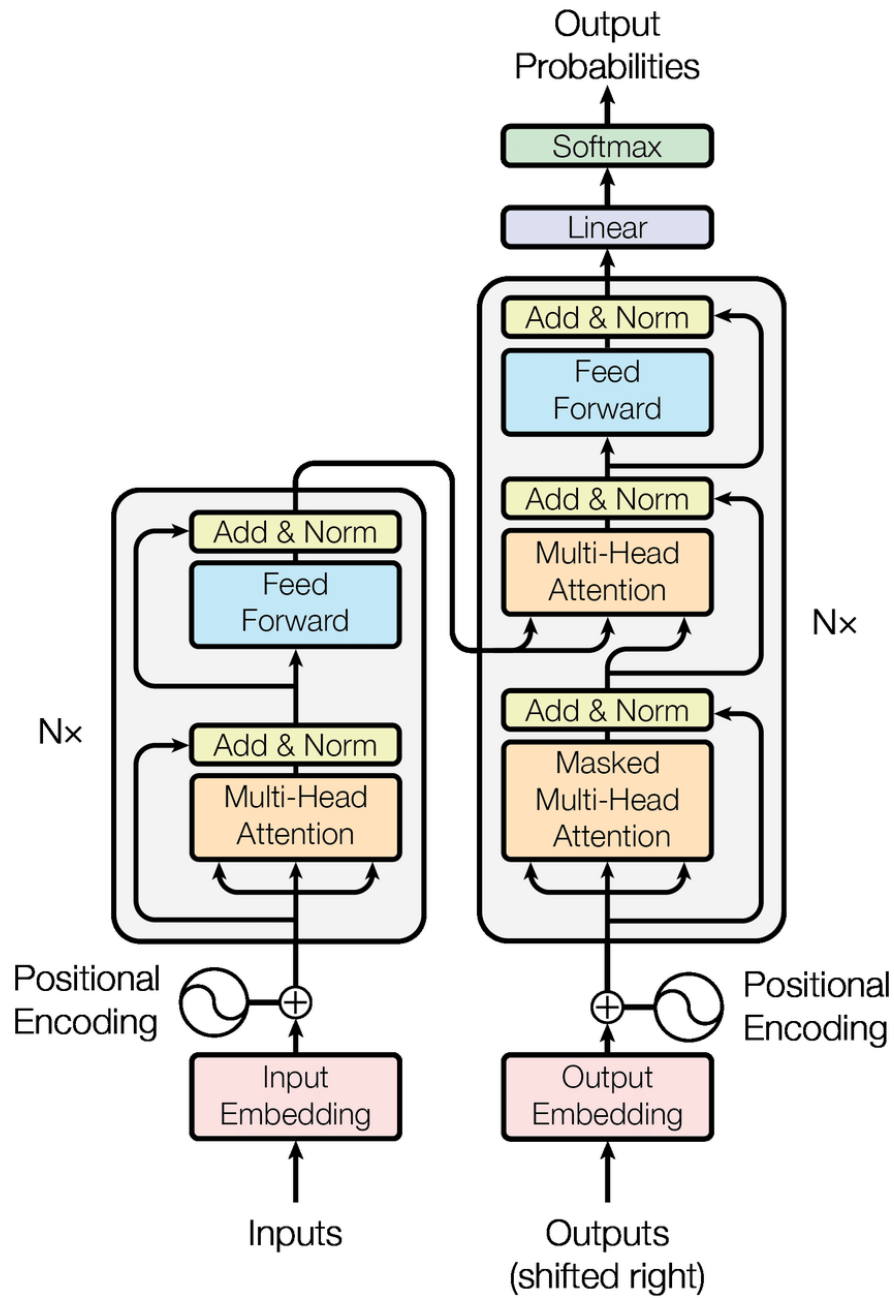
Menggunakan beberapa lapisan RNN sehingga model dapat mempelajari representasi yang lebih kompleks

II.5.2 Transformer

Transformer merupakan arsitektur pemrosesan sekuensial yang tidak mengandalkan koneksi berulang (*recurrent connections*) seperti pada RNN. Arsitektur ini memanfaatkan mekanisme perhatian (*attention*) sebagai komponen utama untuk memodelkan hubungan antar elemen tanpa harus memprosesnya secara serial sehingga dapat mempercepat pelatihan melalui paralelisasi. (Vaswani et al. 2017).

II.5.2.1 Core Transformer Architecture

Core Transformer terdiri dari dua komponen utama: *encoder* dan *decoder*. *Encoder* bertugas mengekstraksi fitur-fitur dari *input*, sementara *decoder* memanfaatkan representasi yang dihasilkan *encoder* untuk memprediksi *output* secara *autoregressive*. Kedua bagian ini menggunakan *positional encoding* dan *attention mechanism* sebagai fondasi pemrosesan data sekuensial. Arsitektur tersebut terlihat pada gambar II.5.8.



Gambar II.5.8: Arsitektur *Transformer* (Vaswani et al. 2017)

1. Positional encoding

Meskipun Transformer tidak memproses data secara berurutan seperti RNN, tetap diperlukan informasi posisi dari token atau data untuk menjaga urutan dalam sekuens. Untuk itu, digunakan *positional encoding* yaitu penambahan *embedding* posisi atau indeks ke *embedding* token. Pendekatan ini memungkinkan model membedakan posisi relatif antar token sehingga hubungan temporal atau urutan data dapat dikonsiderasi meskipun data

$$\text{PE}_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad \text{PE}_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (\text{II.6})$$

Persamaan II.6: Persamaan *positional encoding*

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (\text{II.7})$$

Persamaan II.7: *Scaled Dot-Product Attention*

diproses secara paralel.

Pada persamaan II.6, pos adalah indeks posisi token dalam sekuens, d_{model} dimensi *embedding*, dan i indeks dimensi. Persamaan ini akan menghasilkan pola sinusoidal yang unik untuk setiap posisi sehingga memungkinkan model untuk mempelajari makna posisi relatif secara implisit.

2. Attention Mechanism

Mekanisme perhatian (Bahdanau, Cho, and Bengio 2015; Luong, Pham, and Manning 2015) memiliki peran penting dalam *Transformer*. Ide utama dari *attention mechanism* adalah membiarkan model memfokuskan perhatian pada elemen-elemen *input* yang paling relevan saat memproses token tertentu. Bentuk paling umum adalah ***Scaled Dot-Product Attention*** pada persamaan II.7.

dengan:

- Q (*query*), K (*key*), dan V (*value*) adalah representasi vektor yang dibentuk dari masukan.
- d_k adalah dimensi dari *key* dan *query*.
- Fungsi softmax memastikan bobot *attention* dijumlahkan menjadi 1 di sepanjang dimensi sekuens.

Multi-head attention adalah ekstensi *attention* dengan menggunakan beberapa set (Q, K, V) secara paralel. Setiap *head* dapat belajar jenis relasi yang berbeda di waktu yang sama dan hasilnya akan digabungkan (*conca-*

$$\mathbf{H}' = \text{LayerNorm}\left(\mathbf{H} + \text{MultiHeadAttention}(\mathbf{H}, \mathbf{H}, \mathbf{H})\right) \quad (\text{II.8})$$

Persamaan II.8: *Residual connection & multi-head self-attention*

$$\mathbf{H}^* = \text{LayerNorm}\left(\mathbf{H}' + \text{FFN}(\mathbf{H}')\right) \quad (\text{II.9})$$

Persamaan II.9: *Residual connection & feed-forward network*

tenate) sebelum diproyeksikan kembali.

3. Arsitektur *encoder-decoder*

Encoder dan *decoder* pada *Transformer* terdiri atas beberapa lapisan (*stacked layers*). Setiap lapisan *encoder* memiliki dua sub-lapisan berikut:

(a) **Multi-head self-attention**

Menerapkan *attention* pada *input* itu sendiri (disebut *self-attention*) agar model dapat memahami hubungan antar token dalam sekuens.

(b) **Feed-forward network**

Sebuah *multi-layer perceptron* (MLP) yang diterapkan secara independen pada setiap posisi token, disebut *position-wise feed-forward network*.

Pada setiap sub-lapisan (*attention* dan *feed-forward*), akan ditambahkan *residual connection* (He et al. 2016) dan *layer normalization* (Ba, Kiros, and Hinton 2016), sehingga lapisan ke- l dapat direpresentasikan sebagai:

$\mathbf{H}$ adalah *input layer*, $\mathbf{H}'$ adalah *output* sub-lapisan *self-attention*, dan $\mathbf{H}^*$ menjadi *output* akhir setelah melalui sub-lapisan *feed-forward*. Proses serupa diterapkan pada *decoder*, dengan perbedaan utama adanya:

- **Masked self-attention** pada *decoder* untuk mencegah model mengambil informasi token masa depan ketika memprediksi token saat ini.
- **Cross-attention** pada *decoder* yang mengambil $\mathbf{H}_{\text{encoder}}^*$ sebagai (K, V) , sementara $\mathbf{h}_{\text{decoder}}$ berperan sebagai Q .

4. Computation Complexity

Salah satu kelemahan Transformer adalah kompleksitas komputasi kuadratik ($O(T^2)$) dengan panjang sekuens sebagai T . Setiap token harus menghitung *attention* terhadap semua token lain dalam sekuens sehingga pada sekuens yang sangat panjang, hal ini bisa menjadi *bottleneck*. Meskipun demikian, paralelisasi tetap membuat Transformer efisien.

II.5.2.2 Temporal Fusion Transformer

Temporal Fusion Transformer (TFT) (Lim et al. 2020) adalah versi arsitektur *Transformer* untuk prediksi deret waktu *multi-horizon*. Tujuan utama TFT adalah memaksimalkan pemanfaatan informasi yang sering kali bersifat heterogen (seperti variabel kualitatif, numerik, serta data yang bervariasi sepanjang waktu).

Data deret waktu umumnya bersifat heterogen, mencakup variabel kategorikal, kontinu, statik, dan bervariasi dari waktu ke waktu. Model harus mampu mengidentifikasi variabel-variabel mana yang paling penting serta memahami relasi di antara variabel-variabel tersebut pada setiap titik waktu. Selain itu, dengan memanfaatkan keunggulan *attention* pada *Transformer*, *Temporal Fusion Transformer* (TFT) dapat menangkap *long-term dependencies* secara paralel, sementara mekanisme *gating* (terinspirasi dari LSTM/GRU) membantu menjaga stabilitas saat mempelajari relasi antar variabel. TFT menggunakan *Variable Selection Networks*, mekanisme *attention*, serta *Gated Residual Networks (GRN)* untuk menyaring dan mengalirkan informasi secara tepat.

1. Variable Selection Network (VSN)

Komponen yang secara eksplisit memilih fitur mana yang paling relevan pada setiap waktu. VSN mengeluarkan koefisien atau *weight* untuk setiap variabel lalu hanya variabel yang bobotnya signifikan yang akan dilanjutkan ke tahap waktu berikutnya.

$$\mathbf{h}^{\text{selected}} = \text{VSN}(\mathbf{X}),$$

di mana $\mathbf{X}$ mewakili kumpulan semua variabel (static, temporal, dsb.), sedangkan $\mathbf{h}^{\text{selected}}$ adalah hasil seleksi variabel.

2. Gated Residual Network (GRN)

Berfungsi untuk memproses *input* terpilih dan memberikan mekanisme *gating* yang dapat mengatur seberapa banyak informasi dilewatkan. GRN memiliki struktur *residual connection*, serupa dengan blok Transformer:

$$\mathbf{h}^{\text{processed}} = \text{GRN}(\mathbf{h}^{\text{selected}}).$$

Kombinasi GRN dan VSN membuat TFT bagus dalam menangani ragam variabel yang kompleks.

3. Multi-Head Attention

Setelah tahap VSN dan GRN, TFT menerapkan mekanisme *attention* (*self-attention* ataupun *cross-attention*) pada representasi tersebut. Dengan demikian, interaksi antar token (waktu) dapat ditangkap secara eksplisit:

$$\mathbf{H}_{att} = \text{MultiHeadAttention}(Q = \mathbf{h}^{\text{processed}}, K = \mathbf{h}^{\text{processed}}, V = \mathbf{h}^{\text{processed}}).$$

4. Output GRN & *Final Projection*

Hasil $\mathbf{H}_{att}$ dapat kembali diproses oleh GRN guna menggabungkan informasi *attention* dengan mekanisme *gating*. Selanjutnya, proyeksi akhir dilakukan untuk menghasilkan *forecast* pada setiap langkah waktu mendatang:

$$\mathbf{Z}_{\text{final}} = \text{GRN}(\mathbf{H}_{att}).$$

Model kemudian memetakan $\mathbf{Z}_{\text{final}}$ ke *output* yang diinginkan.

Salah satu kelebihan utama TFT adalah kemampuannya untuk *memilih variabel* secara otomatis melalui *Variable Selection Network* (VSN), sehingga model hanya fokus pada variabel yang paling relevan. Selain itu, *attention* memungkinkan penangkapan *long-term dependencies* dalam data deret waktu tanpa harus memproses token secara serial sehingga dapat mengakses setiap posisi (waktu) sekaligus. Mekanisme *gating* (seperti pada *Gated Residual Network*) berperan penting dalam menambah stabilitas model, membantu mengurangi risiko *overfitting* dan sekaligus memperkuat *feature representation* yang dihasilkan.

Di sisi lain, kompleksitas arsitektur TFT meningkat akibat penggabungan VSN, *Gated Residual Networks*, dan *multi-head attention*, yang secara keseluruhan menambah jumlah parameter dan membutuhkan sumber daya komputasi lebih besar. Meskipun mekanisme *attention* lebih mudah diparalelkan daripada RNN,

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (\text{II.10})$$

Persamaan II.10: *Root Mean Squared Error*

kompleksitas kuadratik $\mathcal{O}(T^2)$ tetap menjadi kendala saat memproses deret waktu berdurasi sangat panjang. Selain itu, konfigurasi *hyperparameter* seperti dimensi *hidden state* dan jumlah *heads* yang tinggi sehingga iterasi sering kali diperlukan demi mengoptimalkan performa model.

II.6 Evaluation Metrics

Dalam penelitian ini, pemodelan *time series* pasar saham Indonesia menggunakan arsitektur *Temporal Fusion Transformer* (TFT) dan *Long Short-Term Memory* (LSTM) beserta variannya akan dievaluasi dengan tiga metrik utama. Ketiga metrik ini dipilih agar dapat menilai baik tingkat kesalahan numerik terhadap harga saham, maupun kemampuan model dalam menangkap pergerakan arah (*direction*) harga. Berikut adalah penjelasan masing-masing metrik.

II.6.1 Root Mean Squared Error (RMSE)

Root Mean Squared Error (RMSE) mengukur seberapa besar perbedaan antara nilai prediksi ($\hat{y}_i$) dan nilai sebenarnya (y_i) pada data *time series*. RMSE dapat dirumuskan dalam Persamaan II.10:

Semakin rendah nilai RMSE, semakin kecil selisih antara prediksi dan nilai aktual. RMSE lebih sensitif terhadap kesalahan yang besar (*large errors*) karena adanya kuadrat pada selisih ($y_i - \hat{y}_i$). RMSE membantu menilai seberapa akurat model memprediksi nilai harga saham dibandingkan dengan harga yang benar-benar terjadi.

II.6.2 Directional Accuracy (DA)

Directional Accuracy (DA) berfokus pada arah pergerakan harga. DA dapat diformulasikan sebagai berikut:

$$DA = \frac{1}{n} \sum_{i=2}^n \mathbb{I}(\text{sign}(y_i - y_{i-1}) = \text{sign}(\hat{y}_i - \hat{y}_{i-1})), \quad (\text{II.11})$$

Persamaan II.11: *Directional Accuracy*

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100\% \quad (\text{II.12})$$

Persamaan II.12: *Mean Absolute Percentage Error*

dengan $\text{sign}(\cdot)$ menyatakan fungsi penanda tanda (positif, negatif), dan $\mathbb{I}(\cdot)$ merupakan fungsi *indicator* yang bernilai 1 jika arah prediksi sama dengan arah nilai aktual, atau 0 jika berbeda. Nilai DA berada dalam rentang $[0, 1]$, di mana nilai 1 menandakan model selalu memprediksi arah pergerakan dengan benar, dan 0 berarti selalu salah.

II.6.3 Mean Absolute Percentage Error (MAPE)

Mean Absolute Percentage Error (MAPE) mengukur kesalahan relatif dalam bentuk persentase, sehingga skala harga saham dapat lebih diperhitungkan. Persamaan tersebut dapat dilihat pada persamaan II.12:

Semakin kecil nilai MAPE, maka semakin baik kualitas prediksi model karena menunjukkan bahwa persentase kesalahan prediksi terhadap nilai aktual semakin kecil. MAPE berguna untuk membandingkan kinerja model pada saham-saham dengan *level price* berbeda, sebab kesalahan relatif ($\frac{|y_i - \hat{y}_i|}{|y_i|}$) dapat menunjukkan seberapa *proportional* kesalahan prediksi di tiap saham.

II.6.4 Coefficient of Determination (R^2)

Coefficient of Determination (R^2) adalah metrik yang dapat mengukur seberapa baik model menjelaskan variasi dari data aktual. R^2 dapat diinterpretasikan sebagai persentase variansi data yang berhasil ditangkap oleh model. Nilai R^2 berada dalam rentang $(-\infty, 1]$, di mana:

- $R^2 = 1$ menandakan model memiliki *perfect fit* (prediksi sama persis dengan nilai sebenarnya).

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (\text{II.13})$$

Persamaan II.13: *Coefficient of Determination* (R^2)

- $R^2 = 0$ berarti model tidak lebih baik daripada sekadar memprediksi nilai rata-rata.
- $R^2 < 0$ dapat terjadi ketika performa model buruk dan tidak mampu menjelaskan variasi data dengan baik.

Persamaan II.13 mendefinisikan R^2 dengan membandingkan *Residual Sum of Squares* (RSS) dan *Total Sum of Squares* (TSS):

dengan

- y_i adalah nilai aktual,
- $\hat{y}_i$ adalah nilai prediksi,
- $\bar{y}$ adalah rata-rata dari nilai aktual ($\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$).

Semakin tinggi nilai R^2 (mendekati 1), maka semakin besar proporsi variansi data aktual yang mampu diprediksi oleh model. Nilai R^2 negatif mengindikasikan bahwa model memprediksi lebih buruk daripada sekadar tebakan nilai rata-rata. Oleh karena itu, R^2 penting untuk menjadi metrik pelengkap, untuk memberikan gambaran seberapa baik keseluruhan variabilitas data dapat dijelaskan oleh model dibandingkan metrik error lain seperti RMSE atau MAPE.

II.7 Penelitian Terkait

II.7.1 *Artificial Neural Network for Predicting Indonesia Stock Exchange Composite Using Macroeconomic Variables* (Alamsyah and Zahir 2018)

Penelitian ini membahas penerapan *Artificial Neural Network* (ANN) untuk memprediksi pergerakan Indeks Harga Saham Gabungan (IHSG) di Bursa Efek Indone-

sia (BEI) dengan memanfaatkan variabel makroekonomi—yakni nilai tukar, suku bunga, inflasi, dan jumlah uang beredar (M2). Data yang digunakan adalah data bulanan dari Desember 2005 hingga November 2017, lalu diolah dengan metode ANN *Backpropagation*. Tujuan utamanya adalah membantu investor memahami pergerakan IHSG dan mengurangi ketidakpastian dalam pengambilan keputusan investasi.

Hasil penelitian menunjukkan bahwa model ANN yang melalui tahap prapemrosesan data dan mempertimbangkan faktor *time delay* tertentu (satu atau dua bulan) mampu memberikan tingkat akurasi prediksi IHSG yang tinggi, dengan nilai akurasi mencapai 96,38% dan error sebesar 0,004. Dengan temuan ini, ditemukan bahwa pentingnya pemilihan variabel makroekonomi yang tepat serta pengolahan data yang cermat untuk menghasilkan model prediksi yang andal.

II.7.2 *Indonesian Banking Stock Price Prediction with LSTM and Random Walk Method* (Heru, Rachmawati, and Suhartono 2021)

Penelitian ini berfokus pada pemodelan dan prediksi harga saham beberapa bank di Indonesia, yaitu PT Bank Rakyat Indonesia Tbk (BBRI), PT Bank Central Asia Tbk (BBCA), dan PT Bank Pembangunan Daerah Jawa Barat dan Banten Tbk (BJBR), menggunakan *metode Long Short-Term Memory* (LSTM) serta *Random Walk* (RW) baik ordo pertama maupun kedua. Data harga historis diperoleh dari *Yahoo Finance* (untuk LSTM) dan file *csv* (untuk RW), lalu dibagi menjadi beberapa skenario panjang periode pelatihan (1–9 tahun). Kinerja model diukur dengan *Root Mean Square Error* (RMSE) dan *Root Mean Square Error of Prediction* (RMSEP).

Hasil eksperimen menunjukkan bahwa LSTM secara konsisten memberikan hasil prediksi yang lebih baik ditunjukkan oleh penurunan nilai RMSE dan RMSEP. Metode RW juga mampu memprediksi pergerakan harga saham meskipun relatif lebih sederhana. Namun, penggunaan *Time-Delay* yang lebih panjang pada LSTM secara umum menghasilkan performa yang unggul. Selain itu, semakin be-

sar ukuran data historis yang digunakan, semakin kecil kesalahan prediksi yang dihasilkan.

BAB III

ANALISIS DAN PERANCANGAN

III.1 Analisis Masalah

Pasar modal Indonesia yang semakin berkembang dan terdiversifikasi menuntut pendekatan analisis serta *forecasting* yang lebih komprehensif. Seperti telah dipaparkan pada Latar Belakang, dinamika harga saham di pasar modal Indonesia dipengaruhi oleh berbagai faktor, mulai dari kebijakan pemerintah, indikator makroekonomi, hingga sentimen publik (Fekrazad, Harun, and Sardar 2022). Selain itu, *financial time series* biasanya bersifat nonstasioner dan mengandung pola-pola yang tidak selalu dapat diwakili dengan hubungan linear semata (Goodfellow, Bengio, and Courville 2016).

III.2 Identifikasi Alternatif Solusi

III.2.1 *Classical Quantitative*

Classical quantitative berfokus pada penggunaan model-model statistik tradisional yang berlandaskan pada asumsi-asumsi linearitas dan stasioneritas data. Metode ini masih banyak digunakan di institusi finansial karena memiliki nilai interpretasi yang tinggi dan landasan teori ekonomi yang kuat.

1. **ARIMA (AutoRegressive Integrated Moving Average)**

ARIMA menjadi metode utama yang sering dipakai dalam *time series forecasting*, terutama ketika data cenderung stasioner atau memiliki pola mu-

siman yang dapat diakomodir dengan baik. Kelebihan ARIMA adalah Koefisien AR (*autoregressive*) dan MA (*moving average*) yang merupakan salah satu indikator dari ARIMA dapat dijelaskan secara statistik, sehingga memudahkan awan dapat memahami model. Namun, ARIMA kurang efektif dalam menangkap hubungan nonlinier antar variabel, dan memerlukan langkah diferensiasi (integrasi) agar data menjadi stasioner. Jika data finansial memiliki banyak variabel eksternal yang saling berkoerlasi, ARIMA menjadi cepat kurang akurat.

2. Regresi Linear

Regresi linear memprediksi sebuah variabel target (*dependent variable*) dari satu atau beberapa variabel lainnya (*independent variables*) dengan asumsi linearitas. Kelebihannya meliputi:

- **Sangat mudah diinterpretasi:** Koefisien regresi menunjukkan seberapa besar dampak setiap variabel input terhadap variabel target.
- **Proses komputasi relatif ringan:** Tidak memerlukan sumber daya besar, sehingga dapat dijalankan dengan cepat.

Kekurangannya adalah model ini tidak dapat mempelajari relasi kompleks atau nonlinier yang kerap muncul dalam data keuangan.

3. Regresi Logistik

Regresi Logistik biasanya digunakan untuk prediksi probabilitas naik atau turunnya harga saham. Pada dasarnya, regresi logistik mengasumsikan linearitas pada $\log\left(\frac{p}{1-p}\right)$ (logit). Model ini tidak dapat menangkap kenonlinearitas data sehingga kerap kali *over-simplifying* pola yang sesungguhnya rumit di pasar modal.

III.2.2 *Sequence-based Modelling*

Sequence-based modelling digunakan untuk mempelajari data sekuensial (*time series*) yang berisikan interaksi jangka panjang dan pola kompleks.

1. *Gated RNN*

LSTM dibuat untuk mengatasi keterbatasan RNN *vanilla*, khususnya masalah *vanishing gradient* (Hochreiter and Schmidhuber 1997). Model LSTM mampu “mengingat” informasi relevan lebih lama karena memiliki struktur *cell state* dan *gates*. Struktur dari *gating* ini dimiliki juga oleh GRU.

- **Kelebihan:** Cocok untuk data dengan dependensi jangka panjang, seperti *time series* finansial. Dapat menemukan *hidden pattern* yang kompleks, bahkan ketika data memiliki faktor musiman dan tren yang berubah-ubah.
- **Kekurangan:** Proses pelatihan lebih lambat jika dibandingkan model linear dan membutuhkan *fine-tuning* hyperparameter yang ekstensif (jumlah *hidden units*, *learning rate*, dsb.). Keterbatasan paralelisasi RNN juga membuat waktu *training* lebih lama pada dataset yang amat besar.

2. Temporal Fusion Transformer (TFT)

TFT (Lim et al. 2020) mengadopsi mekanisme *attention* dari *Transformer* (Vaswani et al. 2017), tetapi dirancang khusus untuk *multi-horizon time series forecasting*.

- **Kelebihan:** Mampu mempelajari dependensi jangka panjang tanpa harus memproses data secara sekuensial serta mendukung paralelisasi yang lebih baik. TFT memiliki modul *variable selection* yang dapat meningkatkan interpretasi model. Hal ini membuat TFT lebih fleksibel untuk menangani data multivariabel yang kompleks dan *non-stationary*.
- **Kekurangan:** Kompleksitas komputasi $\mathcal{O}(T^2)$ dapat menjadi masalah ketika panjang sekuens T sangat besar, dan proses *training* memerlukan sumber daya yang signifikan (misalnya GPU atau TPU).

III.2.3 Classical Machine Learning

Kategori *classical machine learning* terdiri atas *machine learning algorithm* yang tidak secara khusus dirancang untuk data sekuensial, tetapi biasa digunakan

untuk berbagai *prediction task*, termasuk *time series forecasting* dengan dilakukannya *additional feature engineering* (misalnya membuat *lag features*, *moving averages*, atau *seasonly encoding*).

1. Random Forest

Random Forest adalah metode *ensemble* yang membentuk banyak pohon keputusan dengan *bagging* (Breiman 2001).

- **Kelebihan:** Memiliki *robustness* tinggi terhadap outlier dan *noise*, serta cenderung memberikan performa stabil tanpa terlalu banyak *tuning*. Dapat memanfaatkan *feature importance* untuk interpretasi peran tiap variabel.
- **Kekurangan:** Kurang efektif dalam mengungkap *long-term dependencies* bila data tak diolah menjadi format *windowed features*. Selain itu, masih terbatas pada pendekatan $\mathcal{O}(n \log n)$ untuk banyak pohon dan interpretasinya tidak sejelas model linear.

2. XGBoost (Extreme Gradient Boosting)

XGBoost merupakan metode *boosting* yang menambahkan pohon-pohon secara iteratif untuk mengurangi *residual error*.

- **Kelebihan:** Biasa menghasilkan prediksi akurat dengan waktu *training* cukup rendah. Mampu menghadapi data berjumlah besar berkat optimasi level pohon.
- **Kekurangan:** Tetap tidak secara implisit memodelkan sekuens waktu. Jika data sangat *non-stationary* atau memiliki pola musiman, dibutuhkan *feature engineering* yang cermat.

III.3 Evaluasi dan Pemilihan Alternatif Solusi

Analisis dilakukan terhadap tiga alternatif yang telah disebutkan untuk mengidentifikasi kelebihan dan kekurangan masing-masing. Setelah itu, akan dilakukan perhitungan menggunakan *weighted scoring matrix* untuk menentukan solusi terbaik yang dapat diterapkan pada permasalahan tugas akhir. Akan dianalisis

kelebihan serta kekurangan alternatif solusi pada tabel III.3.1.

Tabel III.3.1: Perbandingan Kelebihan dan Kekurangan Alternatif Solusi

Alternatif Solusi	Kelebihan	Kekurangan
Classical Quantitative (ARIMA)	• Koefisien AR dan MA mudah diinterpretasi secara statistik • Bagus untuk data stasioner dan <i>seasonal pattern</i> • Proses komputasi lebih cepat	• Tidak efektif untuk mencari hubungan nonlinier antar variabel • Memerlukan diferensiasi untuk data non-stasioner • Akurasi menurun saat ada banyak variabel yang berkorelasi
Sequence-based Modelling (Gated RNN, TFT)	• Bagus dalam menangkap pola jangka panjang • Mampu menangani data nonlinier dan <i>seasonal pattern</i> • Dapat menemukan <i>hidden pattern</i> yang kompleks	• Waktu <i>training</i> lebih lama • Membutuhkan <i>fine-tuning hyperparameter</i> • Perlu sumber daya komputasi besar
Classical ML (Random Forest, XGBoost)	• <i>Robust</i> terhadap <i>outlier</i> dan <i>noise</i> • Performa stabil tanpa memerlukan banyak <i>tuning</i> • Dapat menunjukkan <i>feature importance</i>	• Perlu feature engineering yang lebih ekstensif • Kurang efektif untuk <i>long-term dependencies</i>

Tabel III.3.2: Weighted Scoring Matrix Alternatif Solusi

Kriteria (Bobot)	Classical Quantitative (ARIMA)	Sequence-based Modeling (RNN, TFT)	Classical ML (RF, XGBoost)
Skalabilitas (20%)	$3 \times 0.2 = 0.60$	$2 \times 0.2 = 0.40$	$2 \times 0.2 = 0.40$
Biaya (5%)	$2 \times 0.05 = 0.10$	$3 \times 0.05 = 0.15$	$2 \times 0.05 = 0.10$
Kesederhanaan (35%)	$2 \times 0.35 = 0.70$	$3 \times 0.35 = 1.05$	$1 \times 0.35 = 0.35$
Waktu Implementasi (25%)	$2 \times 0.25 = 0.50$	$3 \times 0.25 = 0.75$	$2 \times 0.25 = 0.50$
Performa Model (15%)	$3 \times 0.15 = 0.45$	$2 \times 0.15 = 0.30$	$2 \times 0.15 = 0.30$
Total Skor	2.35	2.65	1.65

Dari tabel III.3.2, solusi yang akan dipilih adalah solusi *Classical Quantitative* (ARIMA) serta *Sequence based model*.

III.4 Perancangan Konsep Desain dan Solusi

III.4.1 Data Preparation

Data yang digunakan dalam penelitian ini berasal dari *Yahoo! Finance API* dan mencakup data historis pasar modal Indonesia, termasuk saham-saham yang terdaftar di Bursa Efek Indonesia (BEI). Dataset ini memiliki cakupan data yang dimulai sejak tahun 2005. Dataset ini terdiri dari beberapa fitur utama, seperti ditampilkan pada Tabel III.4.1.

Tabel III.4.1: Metadata of Stock Market Time Series Dataset

Nama Atribut	Tipe Data	Deskripsi
Date	datetime64	<i>identifier</i> unik untuk tanggal perdagangan.
Ticker	string	Simbol <i>ticker</i> saham .
Stock Name	string	Nama lengkap saham .
Open	float64	Harga pembukaan saham pada hari tersebut.
High	float64	Harga tertinggi saham pada hari tersebut.
Low	float64	Harga terendah saham pada hari tersebut.
Close	float64	Harga penutupan saham pada hari tersebut.
Adj Close	float64	Harga penutupan yang disesuaikan (apabila ada <i>event</i> khusus seperti stock split).
Volume	int64	Volume saham yang diperdagangkan pada hari tersebut.

- **Date:** Tanggal perdagangan yang digunakan sebagai penanda unik untuk setiap titik waktu dalam *dataset*. Fitur ini merepresentasikan waktu terjadinya aktivitas perdagangan *time series*.
- **Ticker:** *String* unik yang merepresentasikan saham tertentu, misalnya BBKA.JK untuk saham Bank BCA. *Ticker* digunakan untuk mengidentifikasi perusahaan tertentu dalam pasar saham.
- **Stock Name:** Nama lengkap saham untuk mempermudah interpretasi data.
- **Open, High, Low, Close:**
 - **Open:** Harga pembukaan saham pada awal sesi perdagangan. Fitur ini memberikan informasi tentang harga pertama yang diperdagangkan pada hari tertentu.
 - **High:** Harga tertinggi saham yang tercapai selama sesi perdagangan. Fitur ini mencerminkan *upperbound* harga saham pada hari tersebut.
 - **Low:** Harga terendah saham yang tercapai selama sesi perdagangan. Fitur ini menggambarkan *lowerbound* harga saham pada hari tersebut.

- **Close:** Harga penutupan saham pada akhir sesi perdagangan.
- **Adj Close:** Harga penutupan yang telah disesuaikan untuk memperhitungkan aksi korporasi seperti *stock split*. Fitur ini sangat berguna untuk analisis historis karena mencerminkan nilai saham yang sebenarnya tanpa bias akibat perubahan struktural dari harga saham sehingga memungkinkan perbandingan harga yang lebih akurat dari waktu ke waktu.
- **Volume:** Jumlah saham yang diperdagangkan selama hari tertentu. Fitur ini mencerminkan tingkat aktivitas perdagangan saham . Volume yang tinggi menunjukkan minat pasar yang besar, sedangkan volume rendah dapat mengindikasikan aktivitas perdagangan yang terbatas.

III.4.2 Data Understanding

Pada bagian *Data Understanding*, akan dilakukan *EDA* terhadap data yang akan digunakan. Data yang digunakan adalah *time series data* dari BBKA.JK. Tabel III.4.2 dan tabel III.4.3 membahas tentang karakteristik statistik dari data tersebut.

Tabel III.4.2: Missing Value Analysis for Stock Market Data

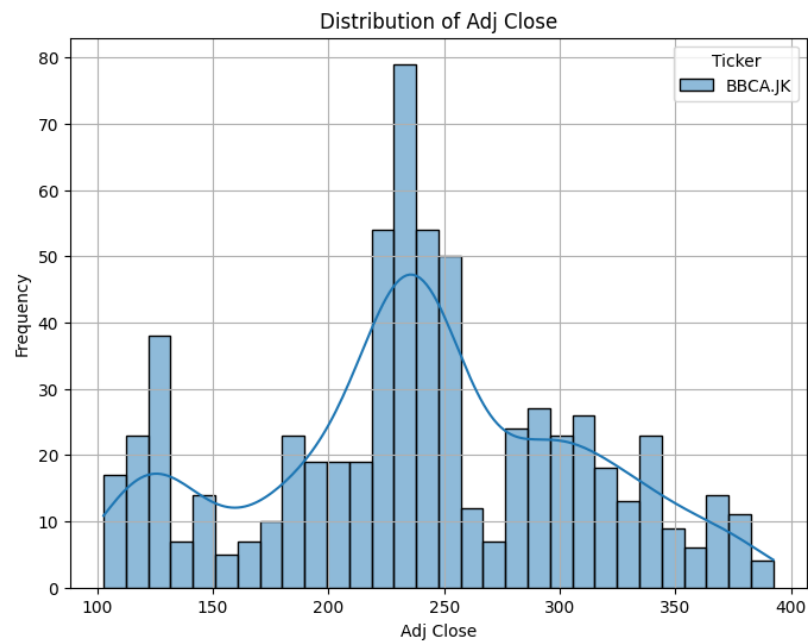
Column	Missing Count	Missing Percentage
Date (Date)	0	0.0%
Adj Close (BBKA.JK)	0	0.0%
Close (BBKA.JK)	0	0.0%
High (BBKA.JK)	0	0.0%
Low (BBKA.JK)	0	0.0%
Open (BBKA.JK)	0	0.0%
Volume (BBKA.JK)	0	0.0%

Tabel III.4.3: Descriptive Statistics for Stock Market Data

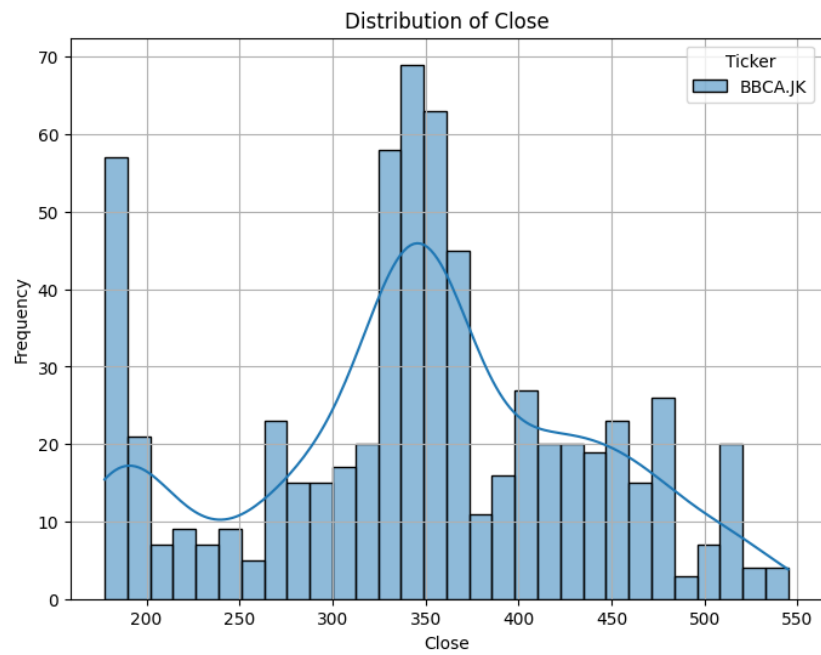
Column	Count	Mean	Std	Min	5%	10%	25%
Open	655	346.39	90.90	175.00	182.50	188.50	295.00
High	655	350.18	91.69	177.50	185.00	191.00	297.50
Low	655	342.16	89.65	175.00	182.50	187.50	292.50
Close	655	346.81	90.56	177.50	185.00	190.00	297.50
Adj Close	655	239.12	69.12	102.76	120.06	125.00	199.41
Volume	655	2.66e+08	2.62e+08	0.00	0.00	4.39e+07	1.13e+08

Column	50%	75%	95%	99%	Max	Skew	Kurtosis
Open	347.50	410.00	500.00	522.30	545.00	-0.16	-0.53
High	352.50	417.50	503.25	535.00	555.00	-0.15	-0.50
Low	342.50	405.00	486.50	517.30	535.00	-0.15	-0.52
Close	347.50	410.00	500.00	527.30	545.00	-0.15	-0.51
Adj Close	236.59	289.83	359.86	379.50	392.24	-0.06	-0.51
Volume	1.91e+08	3.37e+08	7.49e+08	1.48e+09	1.95e+09	2.74	10.78

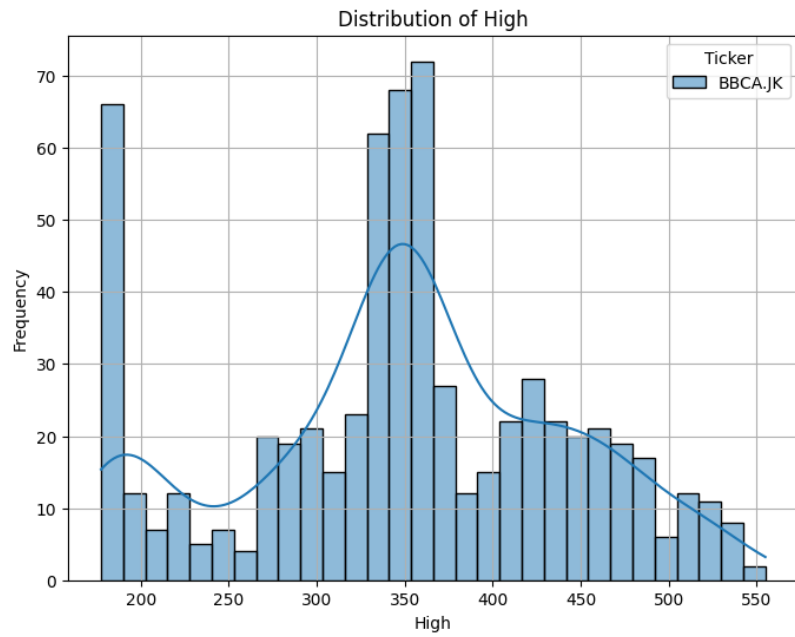
Gambar III.4.1 menggambarkan distribusi persebaran nilai *adj. close*. Gambar III.4.2 menggambarkan distribusi persebaran nilai *closing price*. Gambar III.4.3 menggambarkan distribusi persebaran nilai *highest price*. Gambar III.4.4 menggambarkan distribusi persebaran nilai *lowest price*. Gambar III.4.5 menggambarkan distribusi persebaran nilai *opening price*. Gambar III.4.6 menggambarkan distribusi persebaran volume penjualan.



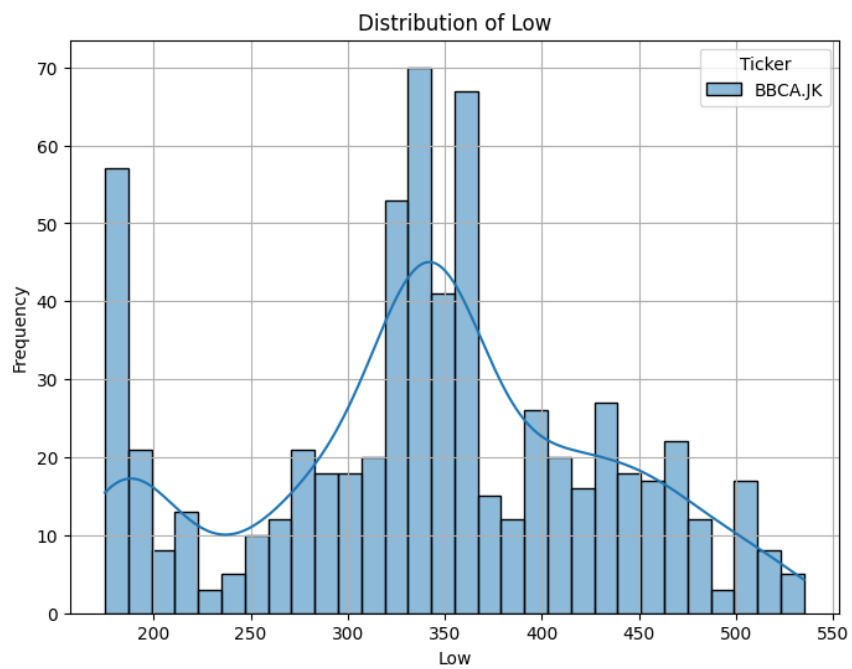
Gambar III.4.1: Distribusi *Adj. Close*



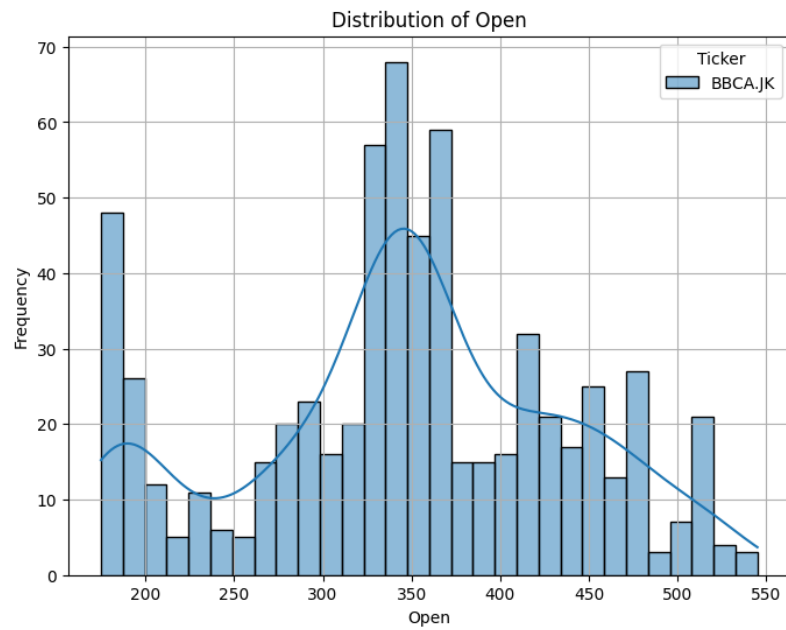
Gambar III.4.2: Distribusi *Closing price*



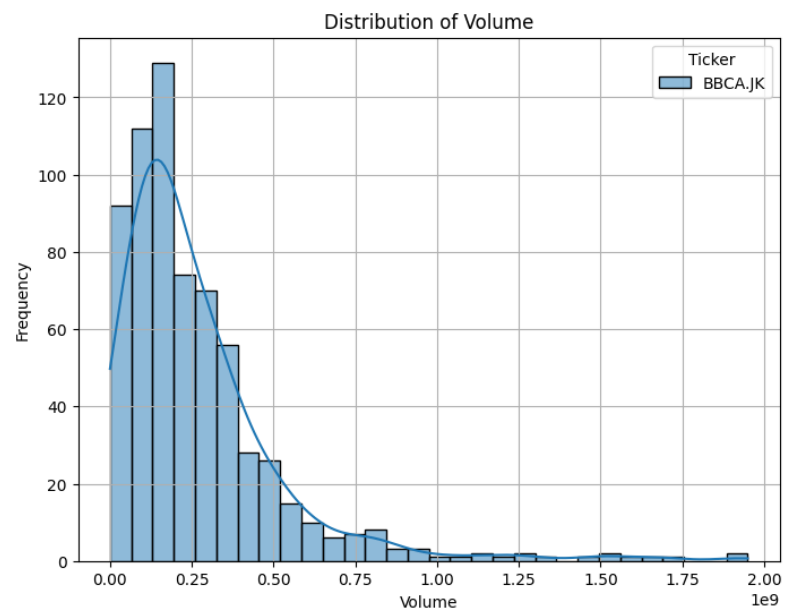
Gambar III.4.3: Distribusi *High price*



Gambar III.4.4: Distribusi *low price*



Gambar III.4.5: Distribusi *opening price*



Gambar III.4.6: Distribusi volume penjualan

III.4.3 Rancangan Konsep Solusi

Berikut adalah uraian lebih detail mengenai Rancangan Konsep Solusi seperti tergambar pada Gambar III.4.7. Rancangan ini secara garis besar terdiri dari

beberapa tahapan utama: *fetching data*, *exploratory data analysis*, *data transformation*, *modeling*, dan *evaluation*. Tiap tahapan saling berkaitan untuk membentuk rangkaian proses *directed graph*, dari pengumpulan data hingga evaluasi performa model.

1. ***Data Fetching***

Pada tahap awal, data diambil (*fetching*) dari *Yahoo! Finance*—sering disebut *YFinance API*. Data ini merupakan *time series* historis berbagai saham di Bursa Efek Indonesia.

2. ***Exploratory Data Analysis (EDA)***

EDA dilakukan untuk mendapatkan pemahaman mendasar tentang struktur dan karakteristik data. Langkah ini meliputi pengecekan *missing values*, deteksi *outliers*, serta analisis *distribution* untuk setiap *feature* (misalnya *close*, *open*, *volume*). Selain itu, kita juga memvisualisasikan pola trend atau *seasonality* pada harga saham. Pada tahap ini, *visual analytics* dan teknik *statistical summarization* berperan penting dalam mencari tahu *possible hidden patterns*.

3. ***Data Transformation***

Data perlu ditransformasikan agar siap dimasukkan ke model. Transformasi dapat mencakup:

- ***Feature Engineering***

Membuat kolom baru seperti *lag features*, *moving averages*, *volatility*, bahkan indikator teknis (misalnya *RSI*, *MACD*, atau *Bollinger Bands*).

- ***Normalization/Scaling***

Menangani perbedaan skala data antar kolom, misalnya menggunakan *MinMaxScaler* atau *StandardScaler* agar model dapat melakukan optimasi parameter dengan lebih stabil.

- ***Splitting Dataset***

Membagi data menjadi *training set*, *validation set*, dan *test set* (atau *out-of-sample*) untuk membuat evaluasi performa model yang lebih *unbiased*.

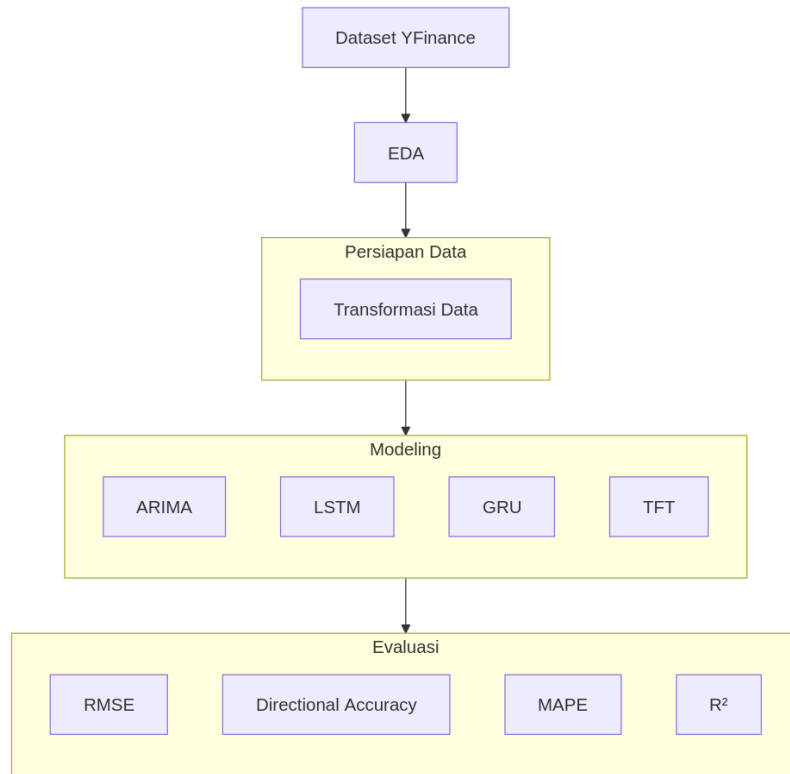
4. *Modeling*

Pada tahap ini, kita memanfaatkan pendekatan *Classical Quantitative* (ARIMA) serta *Sequence-based Modeling* (LSTM, GRU, TFT) yang sudah dipilih pada alternatif solusi untuk membangun model *forecasting*. Pada fase ini, dilakukan juga *hyperparameter tuning* untuk memastikan model bekerja pada konfigurasi optimal (misalnya jumlah *hidden units*, *learning rate*, jumlah *Transformer heads*, dan lain-lain). Proses *training* umumnya dijalankan menggunakan *mini-batch stochastic gradient descent* (*SGD*) atau varian optimisasi populer (*Adam*, *RMSPProp*), disertai mekanisme *early stopping* demi menghindari *overfitting*.

5. *Evaluation*

Setelah model dilatih, performanya dinilai menggunakan beberapa metrik, yaitu:

- **RMSE (Root Mean Squared Error)**: Mengukur *magnitude* kesalahan, sensitif terhadap *outliers*.
- **Directional Accuracy**: Menilai kemampuan model dalam memprediksi arah pergerakan harga (naik/turun).
- **MAPE (Mean Absolute Percentage Error)**: Mengukur kesalahan relatif dalam bentuk persentase, cocok untuk membandingkan saham dengan skala harga berbeda.
- **R^2 (Coefficient of Determination)**: Mengecek seberapa besar variasi data dijelaskan oleh model.



Gambar III.4.7: Diagram Konsep Solusi

BAB IV

RENCANA SELANJUTNYA

IV.1 Rencana Implementasi

IV.1.1 Perkakas

Tabel IV.1.1 akan menjelaskan kakas apa saja yang akan digunakan dalam penelitian nantinya.

Tabel IV.1.1: Spesifikasi Perangkat

Kakas	Spesifikasi
Laptop	CPU: Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz GPU: NVIDIA GeForce GTX 1650 Mobile Ukuran memori: 8 GB
Google Colab Pro	Peladen: NVIDIA Tesla T4 Ukuran memori: 12 GB GPU VRAM: 16 GB
Visual Studio Code	Sistem operasi: Ubuntu 24.04

IV.1.2 Biaya

Tabel IV.1.2 akan menjelaskan biaya apa saja yang akan dikeluarkan dalam penelitian nantinya.

Tabel IV.1.2: Rincian Biaya

Unit	Biaya
Google Colab Pro	\$10/bulan
Total	\$10/bulan

IV.1.3 Rencana Implementasi

Rencana implementasi terdiri dari 2 tahap yang mencakup *EDA*, *modelling*, evaluasi, dan finalisasi laporan tugas akhir. Rencana implementasi dapat dilihat pada tabel IV.1.3 dan tabel IV.1.4.

Tabel IV.1.3: Linimasa pengerjaan tahap 1

Kegiatan	Januari				Februari				Maret				April			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
<i>EDA</i>																
<i>Modeling</i>																
Evaluasi																

Tabel IV.1.4: Linimasa pengerjaan tahap 2

Kegiatan	Mei				Juni			
	1	2	3	4	1	2	3	4
Finalisasi laporan akhir								
Sidang laporan tugas akhir								

IV.2 Desain Pengujian dan Evaluasi

Training dari model akan dilakukan untuk semua *timepoint* kecuali dalam rentang 1 bulan terbaru. Semua data dalam 1 bulan terakhir akan digunakan untuk menjadi *validation data*. Setiap model akan di-*training* menggunakan data *training* yang sama. Metrik metrik yang digunakan adalah RMSE, DA, MAPE, dan R^2 . Setelah itu, setiap model akan di-*sort* performanya berdasarkan keempat metrik tersebut.

IV.3 Analisis Mitigasi dan Risiko

IV.3.1 *Timeline* yang Terlewat

Ada risiko bahwa akan ada tahapan memakan waktu lebih lama dari yang direncanakan. Hal ini dapat menyebabkan keterlambatan dalam penyelesaian keseluruhan proyek, terutama jika waktu yang dialokasikan untuk setiap tahap tidak mencukupi. Untuk mengatasi risiko ini, diperlukan perencanaan jadwal yang fleksibel. Monitoring progres secara berkala juga harus dilakukan menggunakan alat seperti Gantt Chart atau aplikasi produktivitas lainnya. Waktu yang tersisa dari fase yang selesai lebih cepat dapat dialokasikan ke fase berikutnya untuk memastikan penyelesaian proyek tetap tepat waktu.

IV.3.2 Hasil Pemodelan yang Kurang Baik

Risiko lainnya adalah model yang dihasilkan memiliki performa yang tidak memuaskan, baik karena *overfitting*, *underfitting*, atau kurangnya fitur-fitur data yang relevan. Oleh karena itu, seiring berjalannya penelitian, diperlukan juga *research* lebih lanjut tentang data-data *time-series* lainnya seperti data nilai tukar rupiah relatif terhadap dollar AS yang dapat di-*coupling* untuk digunakan untuk melakukan *training* model lebih lanjut.

IV.3.3 Pemodelan Memakan Waktu yang Cukup Lama

Pelatihan model dengan arsitektur kompleks seperti *Gated Recurrent Neural Networks* (GRU dan LSTM) serta *Temporal Fusion Transformer* (TFT) sering kali membutuhkan *training time*. Untuk mengatasi tantangan ini, beberapa strategi optimasi akan dilakukan. Salah satunya adalah penggunaan metode *random search* yang dapat melakukan eksplorasi lebih luas terhadap kombinasi *hyperparameter* tanpa harus mencoba setiap konfigurasi seperti dalam *grid search*. Metode ini sering kali lebih efisien untuk *dataset* besar.

Selain itu, pendekatan *Bayesian optimization* juga dapat dikonsiderasi di mana proses *tuning* dilakukan secara iteratif dengan memanfaatkan hasil sebelumnya

untuk memperkirakan konfigurasi yang optimal. Hal ini lebih efektif dalam menemukan kombinasi *hyperparameter* terbaik dengan jumlah iterasi yang lebih sedikit.

Untuk mempercepat proses pelatihan, layanan *cloud computing* seperti Google Colab Pro akan digunakan guna memanfaatkan GPU atau TPU. Proses pelatihan juga akan dilengkapi dengan fitur *checkpointing* sehingga progres model dapat disimpan dan dilanjutkan jika terjadi gangguan. Untuk TFT, fokus akan terdapat pada pemilihan parameter seperti *learning rate*, jumlah *attention heads*, dan ukuran *hidden layers*, guna memastikan *balance* antara efisiensi waktu dan performa model. Dengan pendekatan ini, waktu pelatihan dapat dioptimisasi secara lebih efektif tanpa mengorbankan kualitas model.

Bibliografi

- Alamsyah, Andry, and Asri Nurfathi Zahir. 2018. "Artificial Neural Network for Predicting Indonesia Stock Exchange Composite Using Macroeconomic Variables". In *2018 6th International Conference on Information and Communication Technology (ICoICT)*, 44–48. <https://doi.org/10.1109/ICoICT.2018.8528774>.
- Asiavesta. 2022. "Development of the Indonesian Capital Market". <https://www.asiavesta.com/en/development-of-the-indonesian-capital-market/>.
- Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. *Layer Normalization*. <https://arxiv.org/abs/1607.06450>. ArXiv e-print. arXiv: 1607.06450 [cs.LG].
- Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. 2015. "Neural Machine Translation by Jointly Learning to Align and Translate". *ICLR*.
- Bhardwaj, P., and J. Kwatra. 2022. "Stock Market Price Forecasting Using Recurrent Neural Network". *Indonesian Journal on Computing (Indo-JC)* 7 (1): 51–60. <https://doi.org/10.34818/INDOJC.2022.7.1.612>.
- Bibi, Iram, Adnan Akhunzada, Jahanzaib Malik, and Sung Won Kim. 2020. "A Dynamic DL-Driven Architecture to Combat Sophisticated Android Malware". https://www.researchgate.net/figure/The-Architecture-of-basic-Gated-Recurrent-Unit-GRU_fig1_343002752.
- Bloom, N., M.A. Kose, F. Ohnsorge, and Z. Yuriy. 2019. "Spillovers of Domestic Shocks: Will they Counterbalance the Global Slowdown?" *World Bank Research Observer*.

- Breiman, Leo. 2001. "Random Forests". *Machine Learning* 45 (1): 5–32. <https://doi.org/10.1023/A:1010933404324>.
- Chatfield, Chris. 2003. *The Analysis of Time Series: An Introduction*. 6th. Chapman / Hall/CRC.
- Cho, Kyunghyun, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. "Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation". *arXiv preprint*, <http://arxiv.org/abs/1406.1078>.
- FAO. 2020. *Food Commodity Markets*. <https://www.fao.org>.
- Fekrazad, Amir, Syed M. Harun, and Naafey Sardar. 2022. "Social Media Sentiment and the Stock Market". *Journal of Economics and Finance* 46. <https://doi.org/10.1007/s12197-022-09575-x>.
- Goetzmann, William N. 2016. *Money Changes Everything: How Finance Made Civilization Possible*. Princeton University Press.
- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press. <http://www.deeplearningbook.org>.
- Haykin, Simon. 1998. *Neural Networks: A Comprehensive Foundation*. 2nd. Upper Saddle River, NJ: Prentice Hall.
- He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. "Deep Residual Learning for Image Recognition". In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.
- Heru, Mike Christ, Ro'fah Nur Rachmawati, and Derwin Suhartono. 2021. "Indonesian Banking Stock Price Prediction with LSTM and Random Walk Method". In *2021 1st International Conference on Computer Science and Artificial Intelligence (ICCSAI)*, 1:385–390. <https://doi.org/10.1109/ICCSAI53272.2021.9609752>.
- Hochreiter, S., and J. Schmidhuber. 1997. "Long Short-Term Memory". *Neural Computation* 9 (8): 1735–1780.

- Hyndman, Rob J., and George Athanasopoulos. 2021. *Forecasting: Principles and Practice*. 3rd. OTexts.
- IMF. 2022. *World Economic Outlook*. <https://www.imf.org>.
- Jobin, Anna, Marcello Ienca, and Effy Vayena. 2019. “The global landscape of AI ethics guidelines”. *Nature Machine Intelligence*.
- Learning, Study Machine. 2021. *Activation Functions in Neural Network*. <https://studymachinelearning.com/activation-functions-in-neural-network/>.
- Lim, Bryan, Sercan O. Arik, Nicolas Loeff, and Tomas Pfister. 2020. “Temporal Fusion Transformers for Interpretable Multi-Horizon Time Series Forecasting”. Version 3, submitted on 27 Sep 2020, *arXiv preprint arXiv:1912.09363*, <https://doi.org/10.48550/arXiv.1912.09363>.
- Luong, Minh-Thang, Hieu Pham, and Christopher D. Manning. 2015. “Effective Approaches to Attention-based Neural Machine Translation”. In *EMNLP*.
- Markham, Jerry W. 2002. *A Financial History of the United States, Volume I: From Christopher Columbus to the Robber Barons (1492-1900)*. M.E. Sharpe.
- Mitchell, Tom M. 1997. *Machine Learning*. McGraw-Hill.
- Mohri, Mehryar, Afshin Rostamizadeh, and Ameet Talwalkar. 2018. *Foundations of Machine Learning*. 2nd. MIT Press.
- Napitupulu, Togar Alam, and Yohanes Budiman Wijaya. 2013. “Prediction of Stock Price Using Artificial Neural Network: A Case of Indonesia”. *Journal of Theoretical and Applied Information Technology* 54 (1): 36–41. https://www.academia.edu/87178436/Prediction_of_Stock_Price_Using_Artificial_Neural_Network_A_Case_of_Indonesia.
- Neal, Larry. 1990. *The Rise of Financial Capitalism: International Markets in the Age of Reason*. Cambridge University Press.
- OECD. 2018. *OECD Economic Surveys: Indonesia 2018*. Organisation for Economic Co-operation and Development. https://doi.org/10.1787/eco_surveys-idn-2018-en.

- OECD. 2020. *OECD Investment Policy Reviews: Indonesia 2020*. <https://www.oecd.org/daf/inv/investment-policy/>.
- OJK. 2007. *Laporan Tentang Penggabungan Bursa Efek Jakarta dan Bursa Efek Surabaya*. Laporan Resmi OJK, 2007.
- Olah, Christopher. 2015. “Understanding LSTM Networks”. <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>.
- Orr, Rebecca B., Neil A. Campbell, Peter V. Minorsky, and Michael L. Cain. 2021. *Biology: A Global Approach*. Pearson.
- Petram, Lodewijk. 2014. *The World’s First Stock Exchange*. Columbia University Press.
- Russell, Stuart, and Peter Norvig. 2020. *Artificial Intelligence: A Modern Approach*. 4th. Prentice Hall.
- S, Ishwarya. November 2024. “Only blog you will need to understand Recurrent Neural Networks (RNNs): A Foundation for Sequence Modeling”. <https://blog.gopenai.com/only-blog-you-will-need-to-understand-recurrent-neural-networks-rnns-a-foundation-for-sequence-8b248a595f9d>.
- Torre, Augusto de la, and Sergio L. Schmukler. 2007. *Emerging Capital Markets and Globalization: The Latin American Experience*. Palo Alto, CA and Washington, DC: Stanford University Press / The World Bank. ISBN: 978-0-8213-6544-1.
- Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, and I. Polosukhin. 2017. “Attention is All You Need”. *Advances in Neural Information Processing Systems* 30.
- Vorhies, William. 2016. “CRISP-DM: A Standard Methodology to Ensure a Good Outcome”. <https://www.datasciencecentral.com/crisp-dm-a-standard-methodology-to-ensure-a-good-outcome/>.

Waskiewicz, Michał. 2021. "Designing Your Neural Networks". Accessed: 2024-12-20. <https://towardsdatascience.com/designing-your-neural-networks-a5e4617027ed?gi=1ec50e0a17bb>.

WB. 2023. *Indonesia Economic Prospects*. <https://www.worldbank.org>.

Yuliani, Isnurhadi, and Ferry Jie. 2017. "Risk Perception and Psychological Behavior of Investors in Emerging Market: Indonesian Stock Exchange". *Investment Management and Financial Innovations* 14 (2). [https://doi.org/10.21511/imfi.14\(2-2\).2017.06](https://doi.org/10.21511/imfi.14(2-2).2017.06).